СПбГУ · МКН · СП · 1 КУРС

Математические основы алгоритмов

Конспект лекций

Лектор: Косолобов Д. А.

Составители: ....

Санкт-Петербург, 2026

Содержание

1 Быстрое преобразование Фурье (FFT). 2

2 Параллельные алгоритмы. Приложения: многопоточность на CPU, цифровая электроника, FPGA/ПЛИС, GPU 6

3 Префиксные суммы (префиксное накопление, операция scan). Сумматор Ладнера–Фишера. 16

4 TODO 21

5 Ранжирование списка во внешней памяти. Линейное программирование 21

6 TODO 35

7 Линейное программирование и метод внутренней точки 35

8 Потоки в сетях. Алгоритм Форда–Фалкерсона. Теорема о максимальном потоке и минимальном разрезе 42

9 TODO 51

10 TODO 51

11 TODO 51

12 TODO 51

13 TODO 51

14 TODO 51

15 TODO 51

Вторая часть курса

1. Быстрое преобразование Фурье (FFT).

Свёртка и умножение многочленов

Определение

Пусть заданы последовательности чисел $a_0, \dots, a_{n-1}$ и $b_0, \dots, b_{n-1}$. Определим последовательность $c_0, \dots, c_{2n-2}$ по формуле

$$c_k = \sum_{t=0}^{n-1} a_t\, b_{k-t}, \qquad k = 0, \dots, 2n-2,$$

(считаем $a_i = b_i = 0$ при $i \notin [0, n-1]$). Эта операция называется **конволюцией (сверткой)**:

$$c = a * b.$$

Умножение многочленов

Рассмотрим многочлены:

$$A(x) = a_0 + a_1 x + \dots + a_{n-1} x^{n-1},$$
$$B(x) = b_0 + b_1 x + \dots + b_{n-1} x^{n-1}.$$

Тогда их произведение:

$$C(x) = A(x)B(x) = c_0 + c_1 x + \dots + c_{2n-2} x^{2n-2},$$

где коэффициенты c_k задаются конволюцией.

Сложность

- Наивное умножение: $O(n^2)$.
- Карацуба: $O\big(n^{\log_2 3}\big)$.
- FFT (быстрое преобразование Фурье): $O(n \log n)$.

Идея FFT-умножения

Хотим быстро вычислить свёртку (а значит и произведение многочленов). Для этого:

1. Выбираем число точек N так, чтобы $N \geq 2n-1$ и обычно $N = 2^t$. Коэффициенты A, B дополняем нулями до длины N.

2. Быстро считаем значения в N точках:

$$A(\omega_k), \quad B(\omega_k), \qquad k = 0, \dots, N-1,$$

   где ω — примитивный N-й корень из единицы.

3. Перемножаем покомпонентно:

$$y_k = A(\omega_k)\, B(\omega_k).$$

4. По значениям y_k восстанавливаем коэффициенты C обратным FFT.

Ключевой факт: можно выбрать точки ω_k так, что и «вычисление значений», и «восстановление коэффициентов» делаются за $O(N \log N)$.

Реализация FFT

Далее считаем, что $N = 2^t$. Если исходная длина не степень двойки, дополняем нулями.

Прямое FFT: из коэффициентов в значения

Выбираем

$$\omega = e^{2\pi i/N} = \cos\frac{2\pi}{N} + i\sin\frac{2\pi}{N}, \qquad \omega_k = \omega^k.$$

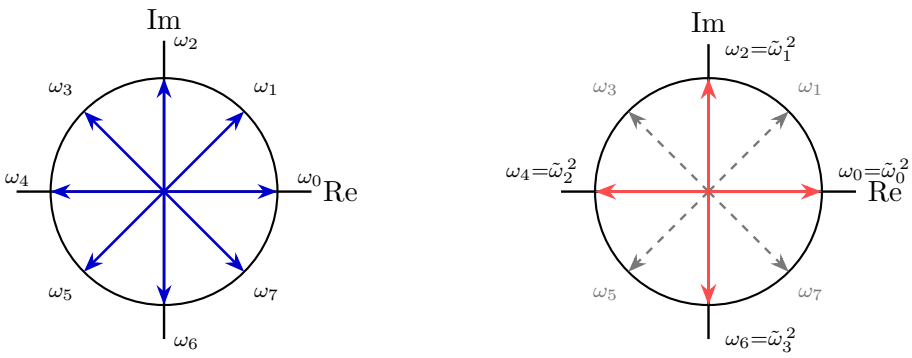


Рис. 1: (слева) Корни из единицы $\omega_k = e^{2\pi ik/8}$ при $N = 8$. При рекурсивном разбиении значения ω^{2k} (справа, красные стрелки) пробегают лишь $N/2$ точек. Серые пунктирные стрелки — нечётные узлы.

Нужно вычислить

$$y_k = A(\omega^k) = \sum_{j=0}^{N-1} a_j (\omega^k)^j, \qquad k = 0, \ldots, N-1.$$

Разобьём A на чётные и нечётные коэффициенты:

$$A_0(x) = a_0 + a_2 x + \cdots + a_{N-2} x^{\frac{N}{2}-1}, \qquad A_1(x) = a_1 + a_3 x + \cdots + a_{N-1} x^{\frac{N}{2}-1}.$$

Тогда

$$A(x) = A_0(x^2) + xA_1(x^2).$$

Подставим $x = \omega^k$:

$$A(\omega^k) = A_0(\omega^{2k}) + \omega^k A_1(\omega^{2k}).$$

Обозначим

$$y_k^{(0)} = A_0(\omega^{2k}), \qquad y_k^{(1)} = A_1(\omega^{2k}).$$

Так как значения ω^{2k} пробегают только $N/2$ различных точек, достаточно посчитать $y_k^{(0)}, y_k^{(1)}$ для $k = 0, \ldots, \frac{N}{2} - 1$ (рекурсивно), а затем «склеить» ответы:

$$y_k = y_k^{(0)} + \omega^k\, y_k^{(1)}, \qquad y_{k+\frac{N}{2}} = y_k^{(0)} - \omega^k\, y_k^{(1)}.$$

(Здесь использовано $\omega^{k+N/2} = -\omega^k$.)

Оценка времени прямого FFT

На каждом уровне рекурсии решаем 2 подзадачи размера $N/2$ и делаем склейку за $O(N)$:

$$T(N) = 2T\left(\frac{N}{2}\right) + O(N) = O(N \log N).$$

Обратное FFT: из значений в коэффициенты

Прямое преобразование можно записать матрично:

$$\begin{pmatrix} 1 & 1 & 1 & \cdots & 1 \\ 1 & \omega^1 & (\omega^1)^2 & \cdots & (\omega^1)^{N-1} \\ 1 & \omega^2 & (\omega^2)^2 & \cdots & (\omega^2)^{N-1} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & \omega^{N-1} & (\omega^{N-1})^2 & \cdots & (\omega^{N-1})^{N-1} \end{pmatrix} \begin{pmatrix} a_0 \\ a_1 \\ \vdots \\ a_{N-1} \end{pmatrix} = \begin{pmatrix} y_0 \\ y_1 \\ \vdots \\ y_{N-1} \end{pmatrix}.$$

Это матрица Вандермонда F_ω.

Утверждение 1.1

Выполнено

$$F_{\omega^{-1}} F_\omega = N \cdot \mathbf{I}.$$

Доказательство. Элемент в позиции (i, j) равен

$$\sum_{k=0}^{N-1} (\omega^i)^k (\omega^{-j})^k = \sum_{k=0}^{N-1} (\omega^{i-j})^k.$$

Если $i \neq j$, то $q := \omega^{i-j} \neq 1$, и по формуле суммы геометрической прогрессии $\sum_{k=0}^{N-1} q^k = \frac{1-q^N}{1-q}$ получаем

$$\sum_{k=0}^{N-1} (\omega^{i-j})^k = \frac{1 - (\omega^{i-j})^N}{1 - \omega^{i-j}} = \frac{1 - (\omega^N)^{i-j}}{1 - \omega^{i-j}} = \frac{1 - 1}{1 - \omega^{i-j}} = 0.$$

Если $i = j$, то

$$\sum_{k=0}^{N-1} (\omega^{i-j})^k = \sum_{k=0}^{N-1} 1 = N.$$

■

Следовательно, обратное преобразование — это прямое FFT с корнем ω^{-1}, после чего нужно поделить все коэффициенты на N: $a = \frac{1}{N} F_{\omega^{-1}} y$.

Сложность умножения многочленов через FFT

Нужно:

- одно прямое FFT для A,
- одно прямое FFT для B,
- покомпонентное умножение ($O(N)$),
- одно обратное FFT.

Итого $O(N \log N)$, где N — ближайшая степень двойки, не меньшая $2n - 1$.

При этом $2n - 1 \leq N < 2(2n - 1) < 4n$, то есть $N = \Theta(n)$, поэтому итоговая сложность равна $O(n \log n)$.

Обобщение: NTT (Number-Theoretic Transform)

Вместо $\mathbb{C}$ можно работать в кольце/поле R, если:

- в R существует элемент ω порядка N (примитивный N-й корень из единицы);
- число N обратимо в R (нужно для деления на N в обратном преобразовании).

Пример (поле $\mathbb{Z}_p$)

Пусть $R = \mathbb{Z}_p$, где p — простое, и

$$N = 2^k, \qquad p = c \cdot 2^k + 1.$$

Тогда $p - 1$ кратно N, и существует первообразный корень g по модулю p (порождает мультипликативную группу порядка $p - 1$). Можно взять

$$\omega \equiv g^c \pmod{p},$$

тогда ω имеет порядок N, и NTT работает полностью по модулю p.

Приложение: смысл коэффициентов DFT

Пусть дан сигнал (последовательность) $a_0, a_1, \ldots, a_{N-1}$.

Определим дискретное преобразование Фурье:

$$y_k = \sum_{t=0}^{N-1} a_t\, \omega^{kt}, \qquad k = 0, \ldots, N-1, \quad \text{где} \;\; \omega = e^{2\pi i/N}.$$

Так как $\omega^{kt} = \cos\left(\frac{2\pi k}{N} t\right) + i \sin\left(\frac{2\pi k}{N} t\right)$,

то

$$\Re(y_k) = \sum_{t=0}^{N-1} a_t \cos\left(\frac{2\pi k}{N} t\right), \qquad \Im(y_k) = \sum_{t=0}^{N-1} a_t \sin\left(\frac{2\pi k}{N} t\right).$$

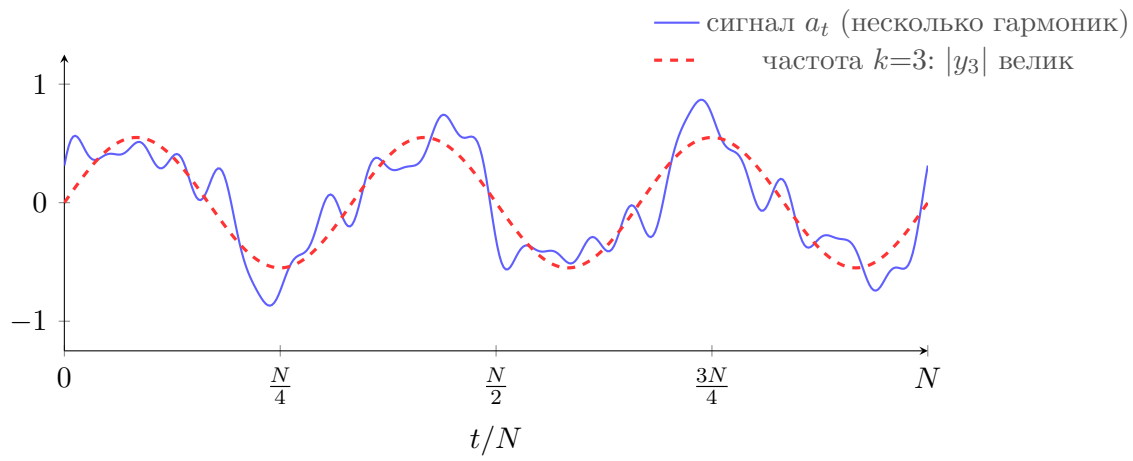


Рис. 2: Если в сигнале (синий) доминирует частота k=3, то коэффициент $y_3 = \sum_t a_t\, e^{2\pi i \cdot 3t/N}$ велик по модулю: вклады $a_t \cos \frac{2\pi \cdot 3\, t}{N}$ складываются (красный пунктир), а для других k — взаимно компенсируются.

Эти суммы — скалярные произведения сигнала с косинусом и синусом частоты k. Если в сигнале сильно выражена гармоника частоты k, то вклады «складываются», и модуль $|y_k|$

получается большим; если такой частоты нет — вклады в основном взаимно компенсируются и $|y_k|$ близок к нулю.

2. Параллельные алгоритмы. Приложения: многопоточность на CPU, цифровая электроника, FPGA/ПЛИС, GPU

2.1. Вычислительные схемы

Определение

Вычислительная схема — это ациклический ориентированный граф, состоящий из:

1. n входных вершин, для которых $\mathrm{indeg}(v) = 0$;
2. m выходных вершин, для которых $\mathrm{outdeg}(v) = 0$;
3. внутренних вершин, каждая из которых помечена элементарной операцией.

В качестве элементарных операций могут выступать арифметические, булевы, а также любые другие операции, считающиеся простыми в данной модели.

Размер схемы — число её внутренних вершин.

Глубина схемы — длина максимального ориентированного пути от входа к выходу.

Замечание

Глубина схемы интерпретируется как параллельное время вычисления: все вершины одного *слоя* (то есть вершины, у которых все входящие данные уже готовы) могут вычисляться одновременно.

Алгоритм в модели вычислительных схем — это семейство схем $\{C_n\}_{n\geq 1}$, где схема C_n решает задачу на входах размера n.

Пример

Простейший пример: схема вычисления $(x_1 + x_2) \cdot (x_3 - x_4)$ имеет четыре входа, два слоя и размер 3.

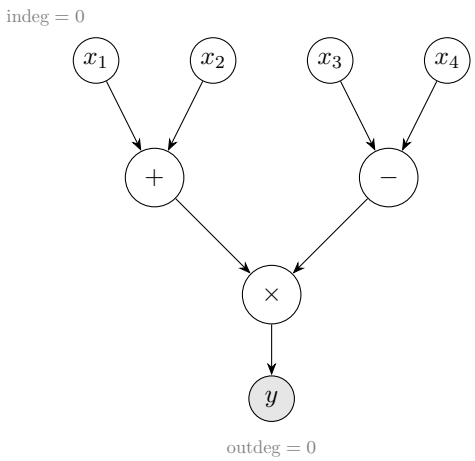


Рис. 3: Не заштрихованные вершины без входящих рёбер — *входы*; заштрихованная вершина без исходящих рёбер — *выход*. **Размер** схемы — 3. **Глубина** — 2.

2.2. Схема быстрого преобразования Фурье

Пусть $n = 2^m$, ω — примитивный корень степени n из единицы, а входной вектор разбит на чётную и нечётную части. Запустив FFT рекурсивно на каждой половине, получаем векторы $y^{(0)}$ и $y^{(1)}$ — результаты DFT для чётных и нечётных компонент соответственно. Тогда объединение (слой «бабочек») имеет вид:

$$y_k = y_k^{(0)} + \omega^k\, y_k^{(1)}, \qquad y_{k+n/2} = y_k^{(0)} - \omega^k\, y_k^{(1)}, \qquad k = 0, 1, \ldots, \tfrac{n}{2} - 1.$$

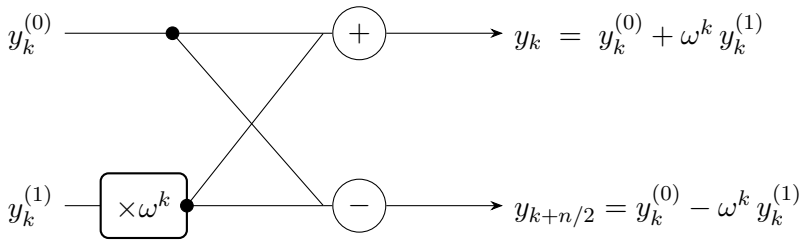


Рис. 4: Одна «бабочка» — элементарный шаг схемы FFT. Нижний входной провод умножается на ω^k; затем два провода попарно складываются и вычитаются. Знак минус объясняется тем, что $\omega^{k+n/2} = -\omega^k$.

Рекурсивная схема FFT состоит из двух блоков $\mathrm{FFT}_{n/2}$, к выходам которых добавляется один слой из $n/2$ бабочек: k-я бабочка умножает правый вход на ω^k, а затем складывает и вычитает.

Сложность

Для схемы FFT выполняются оценки:

$$\text{глубина} = O(\log n), \qquad \text{размер} = O(n \log n).$$

Глубина удовлетворяет рекуррентности $T(n) = T(n/2) + O(1)$, откуда $T(n) = O(\log n)$. Размер удовлетворяет $S(n) = 2S(n/2) + O(n)$, откуда по мастер-теореме $S(n) = O(n \log n)$.

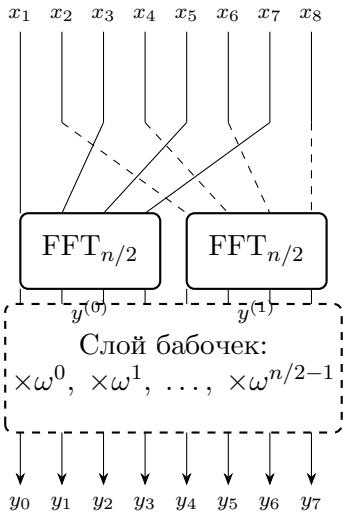


Рис. 5: Рекурсивная схема FFT: нечётные входы поступают в левый блок $\mathrm{FFT}_{n/2}$, чётные — в правый; затем слой бабочек объединяет результаты.

Предложение 2.1

Пусть $y = F_\omega a$, где
$$F_\omega = (\omega^{jk})_{j,k=0}^{n-1},$$
а ω — примитивный корень степени n из единицы. Тогда
$$F_\omega^2 = nP,$$
где P — перестановка $P(a_0, a_1, \ldots, a_{n-1}) = (a_0, a_{n-1}, a_{n-2}, \ldots, a_1)$.

Доказательство. Для любого $a = (a_0, \ldots, a_{n-1})^T$ и любого $i = 0, \ldots, n-1$ имеем
$$(F_\omega^2 a)_i = \sum_{j=0}^{n-1} \omega^{ij} \sum_{k=0}^{n-1} \omega^{jk} a_k = \sum_{k=0}^{n-1} a_k \sum_{j=0}^{n-1} \omega^{j(i+k)}.$$
Но
$$\sum_{j=0}^{n-1} \omega^{jt} = \begin{cases} n, & t \equiv 0 \pmod{n}, \\ 0, & t \not\equiv 0 \pmod{n}. \end{cases}$$
Следовательно,
$$(F_\omega^2 a)_i = n\, a_{-i \bmod n}.$$
Итак, $\omega^2 a = n\,(a_0, a_{n-1}, a_{n-2}, \ldots, a_1)^T = nPa$, то есть $F_\omega^2 = nP$.

Отсюда, если $y = F_\omega a$ и $\tilde{a} = F_\omega y$, то
$$\tilde{a} = F_\omega^2 a = nPa, \qquad a = \frac{1}{n} P\tilde{a}.$$
■

Итак, чтобы восстановить a по y, достаточно:

1. вычислить $\tilde{a} = F_\omega y$;
2. обратить порядок координат $\tilde{a}_1, \ldots, \tilde{a}_{n-1}$, оставив $\tilde{a}_0$ на месте;
3. разделить все координаты на n.

2.3. Сортирующие сети

Определение

Сортирующая сеть — это схема, которая по входу из n чисел $x_1, \ldots, x_n$ выдаёт числа $y_1, \ldots, y_n$, упорядоченные по неубыванию, и использует в качестве внутренних элементов только *компараторы*.

Определение

Компаратор с входами u, v и выходами u', v' вычисляет
$$u' = \min\{u, v\}, \qquad v' = \max\{u, v\}.$$

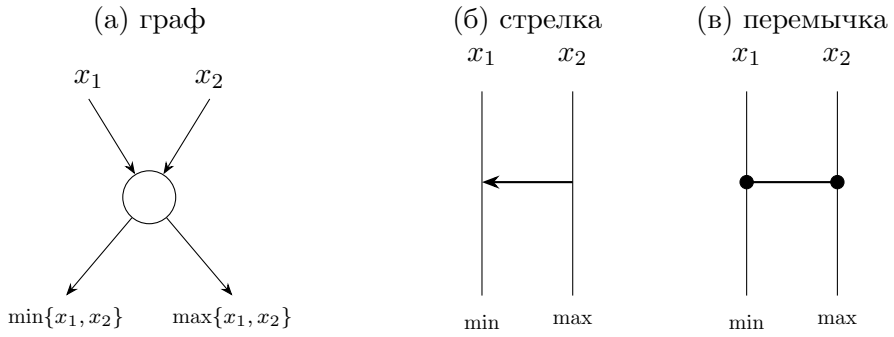


Рис. 6: Три способа изображения компаратора. В обозначениях (б) и (в) сигнал течёт сверху вниз по вертикальным проводам; горизонтальная перемычка сравнивает два соседних элемента и кладёт меньший на левый (верхний) провод.

Нижние оценки

- **Размер** любой сортирующей сети не менее $\Omega(n \log n)$. Поскольку каждый компаратор порождает не более 2 различных исходов, а число различных упорядочений n элементов равно $n!$, то для различения всех входов нужно не менее $\log_2(n!) = \Omega(n \log n)$ компараторов.
- **Глубина** любой сортирующей сети не менее $\Omega(\log n)$. В схеме глубины d значение одного выходного провода может зависеть не более чем от 2^d входных проводов. Так как максимальный элемент на выходе должен зависеть от всех n входов, получаем $2^d \geq n$, откуда $d \geq \log_2 n$.

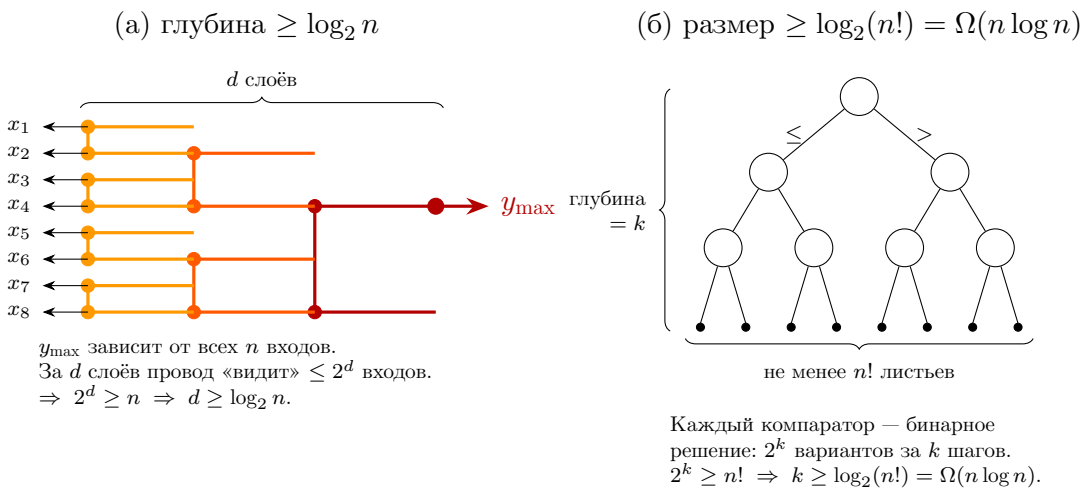


Рис. 7: (а) Нижняя оценка глубины $\Omega(\log n)$: за d слоёв провод максимума может «видеть» не более 2^d входов, а он должен видеть все n. (б) Нижняя оценка размера $\Omega(n \log n)$: нужно различить $n!$ порядков.

Замечание

Существуют сортирующие сети глубины $O(\log n)$ и размера $O(n \log n)$ (сеть AKS). Конструкция весьма сложна, поэтому далее рассматривается более простая *битонная сортировка*.

2.4. Принцип нулей и единиц

Теорема 2.1

Сортирующая сеть корректна на всех вещественных входах тогда и только тогда, когда она корректна на всех входах из нулей и единиц.

Доказательство. Необходимость очевидна: $\{0,1\}$-входы являются частным случаем вещественных.

Докажем достаточность. Предположим, что сеть корректно сортирует все $\{0,1\}$-входы, но не сортирует некоторый вещественный вход $x = (x_1, \ldots, x_n)$. Тогда на выходе получается последовательность $y = (y_1, \ldots, y_n)$, для которой существуют индексы $i < j$ такие, что $y_i > y_j$.

Выберем число t так, что $y_j < t \leq y_i$, и определим булевый вход

$$\widetilde{x}_k = \begin{cases} 0, & x_k < t, \\ 1, & x_k \geq t. \end{cases}$$

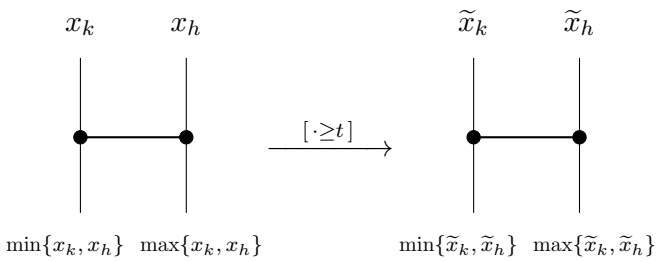


Рис. 8: Применение пороговой функции $[\,\cdot \geq t\,]$ до компаратора даёт тот же результат, что применение после: $[\min\{x_k, x_h\} \geq t] = \min\{[\,x_k \geq t\,], [\,x_h \geq t\,]\}$ и аналогично для max.

Ключевое наблюдение. Пороговая функция $[\cdot \geq t]$ коммутирует с каждым компаратором. Действительно, рассмотрим компаратор на входах u, v, вычисляющий $(\min\{u,v\}, \max\{u,v\})$:

- Если $u, v < t$, то min и max тоже $< t$; после порогования: $(0,0)$. Пороговые значения $(\widetilde{u}, \widetilde{v}) = (0,0)$; компаратор на них даёт $(0,0)$. Совпадает.
- Если $u, v \geq t$, то min и max тоже $\geq t$; после порогования: $(1,1)$. $(\widetilde{u}, \widetilde{v}) = (1,1)$; компаратор на них даёт $(1,1)$. Совпадает.
- Если $u < t \leq v$ (или наоборот), то $\min\{u,v\} < t$ и $\max\{u,v\} \geq t$; после порогования: $(0,1)$. $(\widetilde{u}, \widetilde{v}) = (0,1)$ (или $(1,0)$); компаратор на них даёт $(0,1)$. Совпадает.

Таким образом, если сеть переводит x в y, то она переводит $\widetilde{x}$ в $\widetilde{y}$, где

$$\widetilde{y}_k = \begin{cases} 0, & y_k < t, \\ 1, & y_k \geq t. \end{cases}$$

Из выбора t следует, что $\widetilde{y}_i = 1$ и $\widetilde{y}_j = 0$, то есть выход $\widetilde{y}$ не отсортирован. Получено противоречие с корректностью сети на $\{0,1\}$-входах. ■

2.5. Битонные последовательности

Определение

Последовательность $z_1, \ldots, z_m$ называется *битонной*, если она является циклическим сдвигом последовательности, которая сначала неубывает, а затем невозрастает.

Пример

В случае $\{0,1\}$-последовательностей битонная последовательность имеет вид

$$\underbrace{0\cdots0}_{i}\,\underbrace{1\cdots1}_{j}\,\underbrace{0\cdots0}_{k} \qquad \text{или} \qquad \underbrace{1\cdots1}_{i}\,\underbrace{0\cdots0}_{j}\,\underbrace{1\cdots1}_{k},$$

где $i, j, k \geq 0$, то есть состоит не более чем из трёх чередующихся блоков.

(а) $\underbrace{0\cdots0}_{i}\,\underbrace{1\cdots1}_{j}\,\underbrace{0\cdots0}_{k}$ (б) $\underbrace{1\cdots1}_{i}\,\underbrace{0\cdots0}_{j}\,\underbrace{1\cdots1}_{k}$

Рис. 9: Серые столбцы соответствуют единицам. Параметры $i, j, k \geq 0$; каждый может быть нулём. После одного слоя компараторов (сравнение z_k и $z_{k+m/2}$) минимумы образуют последовательность типа (а), а максимумы — типа (б), либо наоборот.

2.6. Битонное слияние

Пусть $m = 2^t$. Для битонной последовательности $z_1, \ldots, z_m$ определим первый слой компараторов:

$$z_k^{\min} = \min\{z_k,\, z_{k+m/2}\}, \qquad z_k^{\max} = \max\{z_k,\, z_{k+m/2}\}, \qquad k = 1, \ldots, \tfrac{m}{2}.$$

Лемма 2.1

Пусть $z_1, \ldots, z_m$ — битонная последовательность. Тогда после одного слоя компараторов:

1. последовательность $(z_1^{\min}, \ldots, z_{m/2}^{\min})$ битонна;
2. последовательность $(z_1^{\max}, \ldots, z_{m/2}^{\max})$ битонна;
3. каждый элемент первой последовательности не превосходит каждого элемента второй.

Более точно, в случае $\{0,1\}$-последовательностей: либо $z^{\min} = (0, \ldots, 0)$ и $z^{\max}$ битонна, либо $z^{\min}$ битонна и $z^{\max} = (1, \ldots, 1)$.

Это позволяет рекурсивно продолжить сортировку: отсортировать обе половины независимо и объединить результаты.

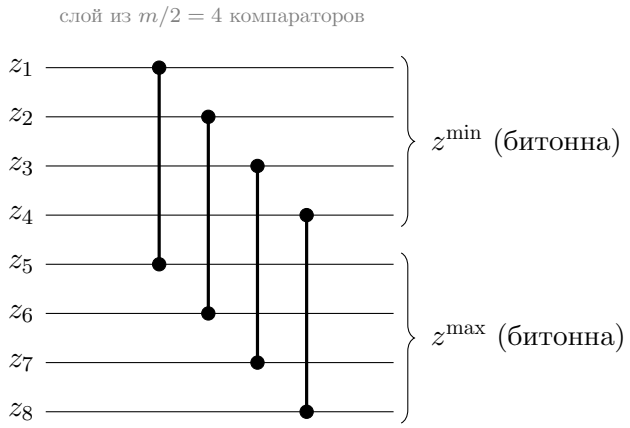


Рис. 10: Первый шаг `bi_merge` при $m = 8$: компаратор k сравнивает z_k и $z_{k+m/2}$. После слоя верхняя половина проводов несёт минимумы, нижняя — максимумы; обе половины являются битонными.

Algorithm 1 BI_MERGE$(z_1, \ldots, z_m)$

Require: Битонная последовательность $z_1, \ldots, z_m$, где $m = 2^t$ для некоторого $t \geq 0$
Ensure: Последовательность $z_1, \ldots, z_m$, отсортированная по неубыванию

```
if m = 1 then
    return (z_1)
for k = 1, ..., m/2 do
    z_k^min = min(z_k, z_{k+m/2})
    z_k^max = max(z_k, z_{k+m/2})
                                        ▷ один слой компараторов
L = BI_MERGE(z_1^min, ..., z_{m/2}^min)   ▷ рекурсивно обрабатываем меньшие элементы
R = BI_MERGE(z_1^max, ..., z_{m/2}^max)   ▷ рекурсивно обрабатываем большие элементы
return L ‖ R                            ▷ конкатенация двух половин
```

Лемма 2.2

Глубина сети `bi_merge` на входе длины m равна $O(\log m)$.

Доказательство. На каждом рекурсивном уровне добавляется один слой компараторов, после чего задача распадается на две независимые подзадачи длины $m/2$. Для глубины $T(m)$:

$$T(m) = T\left(\tfrac{m}{2}\right) + O(1), \qquad T(m) = O(\log m).$$

■

Лемма 2.3

Размер сети `bi_merge` на входе длины m равен $O(m \log m)$.

Доказательство. На верхнем уровне используется $m/2$ компараторов, после чего возникают две подзадачи длины $m/2$:

$$S(m) = 2S\left(\tfrac{m}{2}\right) + O(m), \qquad S(m) = O(m \log m).$$

■

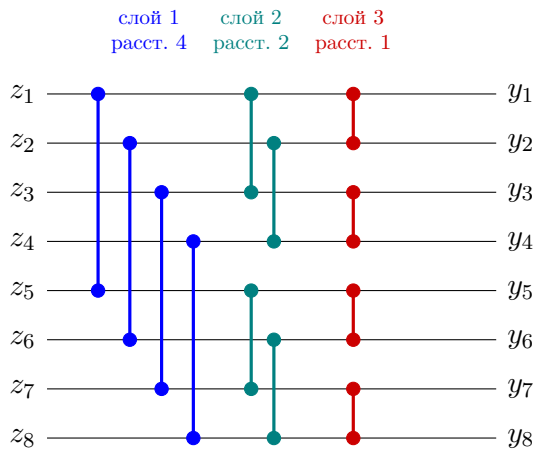


Рис. 11: Полная сеть `bi_merge` для $m = 8$: $\log_2 8 = 3$ слоя. Слой i сравнивает пары на расстоянии $m/2^i$. Глубина $O(\log m)$, размер $O(m \log m)$.

2.7. Битонная сортировка

Предполагается, что $n = 2^t$; если это не так, вход дополняется значением $+\infty$ до ближайшей степени двойки.

Algorithm 2 BITONIC_SORT$(x_1, \ldots, x_n)$

Require: Последовательность $x_1, \ldots, x_n$, где $n = 2^t$ для некоторого $t \geq 0$
Ensure: Последовательность из тех же элементов, отсортированная по неубыванию

```
if n = 1 then
    return (x_1)
(a_1, ..., a_{n/2}) = BITONIC_SORT(x_1, ..., x_{n/2})   ▷ левая половина сортируется по неубыванию
(b_1, ..., b_{n/2}) = BITONIC_SORT(x_{n/2+1}, ..., x_n)   ▷ правая половина сортируется по неубыванию
return BI_MERGE(a_1, ..., a_{n/2}, b_{n/2}, ..., b_1)   ▷ получаем битонную последовательность и сливаем её
```

Конкатенация $(a_1, \ldots, a_{n/2}, b_{n/2}, \ldots, b_1)$ является битонной: первая половина возрастает, вторая убывает.

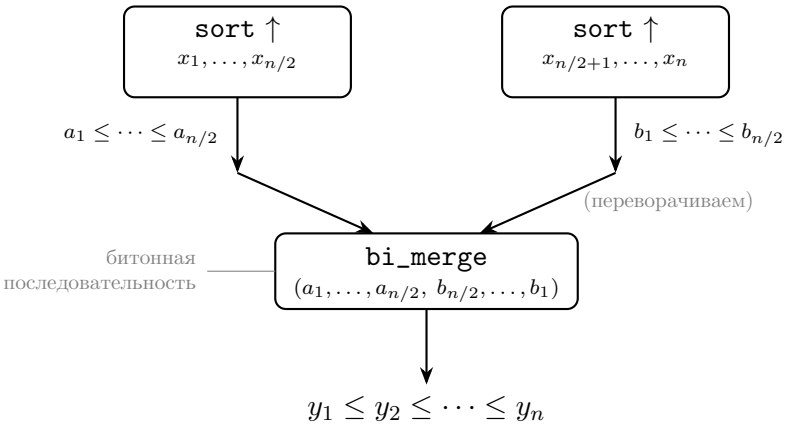


Рис. 12: Рекурсивная структура `bitonic_sort`: обе половины рекурсивно сортируются по возрастанию; правая половина переворачивается, и конкатенация $(a_1, \ldots, a_{n/2},\ b_{n/2}, \ldots, b_1)$ становится битонной последовательностью, которую передают в `bi_merge`.

Замечание

Корректность битонной сортировки на всех вещественных входах вытекает из теоремы о принципе нулей и единиц. Для $\{0,1\}$-входа достаточно убедиться, что все n одинаковых элементов и блоки $0\cdots0\,1\cdots1$ обрабатываются правильно, что проверяется по лемме 2.1.

Теорема 2.2

Битонная сортировка имеет глубину $O(\log^2 n)$ и размер $O(n\log^2 n)$.

Доказательство. Обозначим через $D(n)$ глубину схемы, а через $S(n)$ — её размер.

Глубина. После рекурсивной сортировки половин (глубина $D(n/2)$) выполняется `bi_merge` глубины $O(\log n)$:

$$D(n) = D\left(\tfrac{n}{2}\right) + O(\log n).$$

Раскрывая рекуррентность:

$$D(n) = \sum_{k=0}^{\log n-1} O\left(\log \tfrac{n}{2^k}\right) = O\left(\sum_{k=0}^{\log n-1} (\log n - k)\right) = O(\log^2 n).$$

Размер. Шаг `bi_merge` имеет размер $O(n\log n)$, поэтому:

$$S(n) = 2S\left(\tfrac{n}{2}\right) + O(n\log n), \qquad S(n) = O(n\log^2 n).$$

■

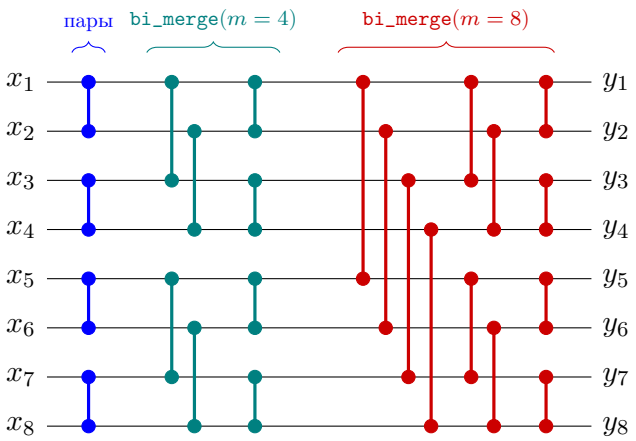


Рис. 13: Полная сортирующая сеть `bitonic_sort` для $n = 8$ ($6 = 1+2+3 = \frac{\log_2 n\,(\log_2 n+1)}{2}$ слоёв). Синие компараторы сортируют пары; зелёные объединяют их в упорядоченные четвёрки; красные выполняют финальное битонное слияние.

Замечание

По сравнению с сетью AKS (глубина $O(\log n)$, размер $O(n\log n)$) битонная сортировка асимптотически хуже, но её конструкция существенно проще и потому она широко используется как базовый пример параллельной сортировки, в частности для реализации на `FPGA` и в `GPU`.

2.8. Нечётно-чётная сортировка (Batcher)

Альтернативный способ построения сортирующей сети с той же асимптотикой — нечётно-чётная сортировка Батчера. Ключевая операция — *нечётно-чётное слияние* двух уже отсортированных последовательностей.

Algorithm 3 ODD_EVEN_MERGE$(a_1, \ldots, a_m;\ b_1, \ldots, b_m)$

Require: Две отсортированные по неубыванию последовательности длины m, где $m = 2^t$
Ensure: Последовательность длины $2m$, полученная слиянием входов и отсортированная по неубыванию

```
if m = 1 then
    u = min(a_1, b_1)
    v = max(a_1, b_1)
    return (u, v)
(c_1, ..., c_m) = ODD_EVEN_MERGE(a_1, a_3, ..., a_{m-1}; b_1, b_3, ..., b_{m-1})   ▷ нечётные позиций
(d_1, ..., d_m) = ODD_EVEN_MERGE(a_2, a_4, ..., a_m; b_2, b_4, ..., b_m)           ▷ чётные позиций
r_1 = c_1
for i = 1, ..., m - 1 do
    r_{2i} = min(c_{i+1}, d_i)
    r_{2i+1} = max(c_{i+1}, d_i)
r_{2m} = d_m                                   ▷ финальный слой из m - 1 компараторов
return (r_1, ..., r_{2m})
```

Лемма 2.4

Процедура ODD_EVEN_MERGE корректно сливает две отсортированные последовательности длины m.

Доказательство. По принципу нулей и единиц рассмотрим вход из $\{0, 1\}$. Докажем корректность индукцией по m.

База: $m = 1$ очевидна.

Шаг: Пусть $m > 1$. По предположению индукции c и d отсортированы. Пусть z_a, z_b — количество нулей во входах. Тогда в c и d их количество равно:

$$z_c = \lceil z_a/2 \rceil + \lceil z_b/2 \rceil, \quad z_d = \lfloor z_a/2 \rfloor + \lfloor z_b/2 \rfloor$$

Так как $0 \le \lceil x/2 \rceil - \lfloor x/2 \rfloor \le 1$, разность $z_c - z_d$ может принимать значения $\{0, 1, 2\}$.

Рассмотрим последовательность $c_1, d_1, c_2, d_2, \ldots, c_m, d_m$. Если $z_c - z_d \in \{0, 1\}$, она уже отсортирована. Если $z_c - z_d = 2$, то c содержит на два нуля больше, чем d. Это создает единственную инверсию $(d_i = 1, c_{i+1} = 0)$ на границе раздела нулей и единиц.

Финальный слой компараторов (d_i, c_{i+1}) для всех i устраняет эту возможную инверсию, делая последовательность полностью отсортированной. ■

Algorithm 4 ODD_EVEN_SORT$(x_1, \ldots, x_n)$

Require: Последовательность $x_1, \ldots, x_n$, где $n = 2^t$ для некоторого $t \geq 0$
Ensure: Последовательность из тех же элементов, отсортированная по неубыванию

```
if n = 1 then
    return (x_1)
(a_1, ..., a_{n/2}) = ODD_EVEN_SORT(x_1, ..., x_{n/2})        ▷ сортировка левой половины
(b_1, ..., b_{n/2}) = ODD_EVEN_SORT(x_{n/2+1}, ..., x_n)      ▷ сортировка правой половины
return ODD_EVEN_MERGE(a_1, ..., a_{n/2}; b_1, ..., b_{n/2})
```

Замечание

Корректность ODD_EVEN_SORT доказывается индукцией по n: сначала рекурсивно сортируются левая и правая половины, после чего они корректно сливаются процедурой ODD_EVEN_MERGE.

3. Префиксные суммы (префиксное накопление, операция scan). Сумматор Ладнера–Фишера.

Задача (prefix sums). Построить вычислительную схему, которая принимает на вход n чисел $x_0, \ldots, x_{n-1}$ и (используя только бинарную операцию "+") выдаёт числа $y_0, \ldots, y_{n-1}$, где

$$y_i = \sum_{j=0}^{i} x_j, \qquad i = 0, \ldots, n-1.$$

Схема Ладнера–Фишера

Пусть n чётное.

Сначала попарно складываем входы:

$$s_i = x_{2i} + x_{2i+1}, \qquad i = 0, \ldots, \frac{n}{2} - 1.$$

Рекурсивно считаем префиксные суммы для s:

$$p_i = \sum_{j=0}^{i} s_j.$$

Тогда выходы восстанавливаются так:

$$y_{2i+1} = p_i, \qquad y_{2i} = \begin{cases} x_0, & i = 0, \\ p_{i-1} + x_{2i}, & i \geq 1. \end{cases}$$

Оценки (глубина и размер).

$$T(n) = 2 + T\left(\frac{n}{2}\right),\ T(1) = 0 \Rightarrow T(n) = 2\log n\ (n = 2^m), \quad S(n) = (n-1) + S\left(\frac{n}{2}\right) = 2n - \log n - 2.$$

Идея хорошо ложится на GPU (много независимых операций на каждом уровне).

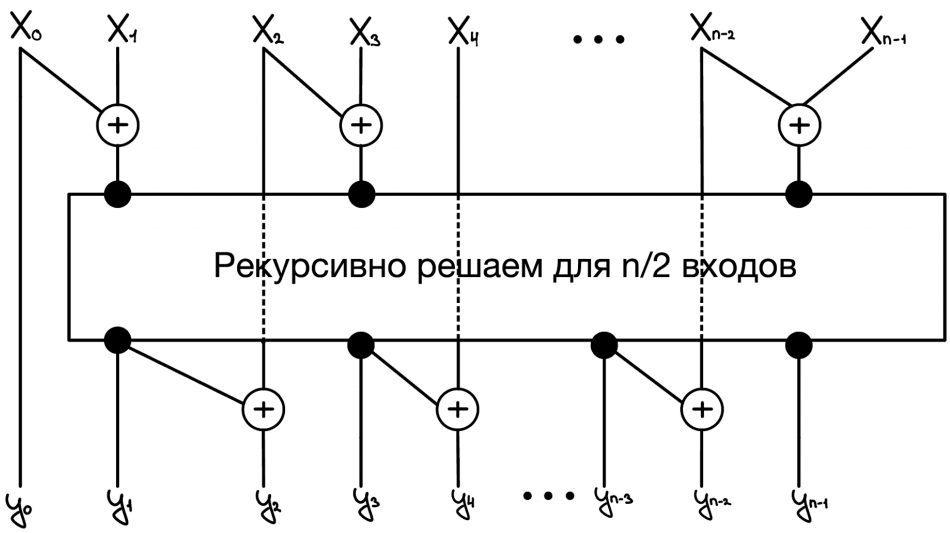


Рис. 14: Схема Ладнера–Фишера для префиксных сумм

Замечание. Операция `scan` (префиксное накопление) определена для любой **ассоциативной** бинарной операции $\odot$ на множестве S (то есть для **полугруппы** $(S, \odot)$):

$$y_i = x_0 \odot x_1 \odot \cdots \odot x_i.$$

Примеры операций $\odot$:

$$(\mathbb{R}, +), \quad (\mathbb{R}, \min), \quad (\mathbb{R}, \max), \quad (\mathbb{R}^{k\times k}, \cdot) \text{ (умножение матриц).}$$

Псевдокод `scan`

Algorithm 5 `scan` (in-place), inclusive, по доске

Require: массив $x[0..n-1]$, где $n = 2^m$

▷ Upsweep: накапливаем суммы на концах блоков

for $d = 0$ **to** $\log_2 n - 1$ **do** ▷ для фиксированного d все операции внутреннего цикла по k выполняются параллельно
  for $k = 0$ **to** $n - 1$ **step** 2^{d+1} **do**
    $x[k + 2^{d+1} - 1] = x[k + 2^{d+1} - 1] + x[k + 2^d - 1]$

▷ Downsweep: добавляем префикс предыдущего блока к левому подблоку

for $d = \log_2 n - 1$ **downto** 0 **do** ▷ для фиксированного d все операции внутреннего цикла по k выполняются параллельно
  for $k = 0$ **to** $n - 1$ **step** 2^{d+1} **do**
    if $k > 0$ **then**
      $x[k + 2^d - 1] = x[k + 2^d - 1] + x[k - 1]$

Пример : Сведения к `scan`: рекурсия вида $c_{k+1} = a_k + b_k c_k$

Даны числа $a_0, \ldots, a_{n-1}$ и $b_0, \ldots, b_{n-1}$. Нужно найти $c_0, \ldots, c_n$ такие, что

$$c_0 = 0, \qquad c_{k+1} = a_k + b_k\, c_k.$$

Определим строковый вектор

$$C_k = (1,\ c_k).$$

Тогда

$$C_{k+1} = C_k\, X_k, \qquad X_k = \begin{pmatrix} 1 & a_k \\ 0 & b_k \end{pmatrix}.$$

Отсюда

$$C_1 = C_0 X_0, \quad C_2 = C_0 X_0 X_1, \quad C_3 = C_0 X_0 X_1 X_2,\ \ldots$$

То есть достаточно посчитать префиксные произведения матриц $X_0, X_1, \ldots, X_{n-1}$ (умножение матриц ассоциативно), и это делается через `scan` за $O(\log n)$ времени в параллельной модели.

Замечание. В примере с матрицами внутри умножения матриц одновременно используются две операции над элементами:

$$(A \cdot B)_{ij} = \bigoplus_k (A_{ik} \odot B_{kj}).$$

Поэтому обобщение работает не для одной операции, а для **полукольца** $(S, \oplus, \odot)$: требуется, чтобы

- $\oplus$ ассоциативна (обычно коммутативна) и имеет нейтральный элемент 0;
- $\odot$ ассоциативна и имеет нейтральный элемент 1;
- $\odot$ дистрибутивна относительно $\oplus$: $a \odot (b \oplus c) = (a \odot b) \oplus (a \odot c)$ и $(a \oplus b) \odot c = (a \odot c) \oplus (b \odot c)$;
- 0 поглощает умножение: $0 \odot a = a \odot 0 = 0$.

Важно: **порядок операций существенен** — в роли "сложения" стоит $\oplus$ (левая операция), а в роли "умножения" стоит $\odot$ (правая операция).

Примеры полукольца $(S, \oplus, \odot)$:

$$(\{0,1\},\ \vee,\ \wedge), \qquad (\{0,1\},\ \oplus,\ \wedge), \qquad (\mathbb{R} \cup \{+\infty\},\ \min,\ +).$$

Сумматор Ладнера–Фишера и переносы

Определение

Сумматор — это вычислительная схема над булевыми операциями $(\vee, \wedge, \oplus, \neg, \ldots)$, которая принимает на вход биты

$$x_0, \ldots, x_{n-1}, \qquad y_0, \ldots, y_{n-1}$$

и выдаёт биты

$$z_0, \ldots, z_n$$

такие, что $z = x + y$, где

$$x = \sum_{k=0}^{n-1} x_k 2^k, \qquad y = \sum_{k=0}^{n-1} y_k 2^k, \qquad z = \sum_{k=0}^{n} z_k 2^k.$$

Биты переноса и сведение к `scan`

Пусть c_k — перенос *в* разряд k (по определению $c_0 = 0$). Тогда для $k = 0, \dots, n-1$ выполняется формула переноса:

$$c_{k+1} = (x_k \wedge y_k) \ \vee \ (x_k \wedge c_k) \ \vee \ (y_k \wedge c_k).$$

Её удобно переписать в виде

$$c_{k+1} = a_k \ \vee \ (b_k \wedge c_k), \qquad a_k := x_k \wedge y_k, \quad b_k := x_k \vee y_k.$$

Если все переносы $c_0, \dots, c_n$ известны, то ответ восстанавливается по формулам

$$z_k = x_k \oplus y_k \oplus c_k \quad (k = 0, \dots, n-1), \qquad z_n = c_n.$$

Ключевая идея: рекуррента

$$c_{k+1} = a_k \ \vee \ (b_k \wedge c_k)$$

имеет вид "аффинного" преобразования, которое можно упаковать в матрицу. Положим

$$C_k = (1, \ c_k), \qquad X_k = \begin{pmatrix} 1 & a_k \\ 0 & b_k \end{pmatrix}.$$

Тогда

$$C_{k+1} = C_k X_k,$$

где операции в матрице понимаются над булевым полукольцом $(\{0,1\}, \ \vee, \ \wedge)$. Следовательно, нужно посчитать префиксные произведения

$$C_k = C_0 X_0 X_1 \cdots X_{k-1},$$

что делается через `scan` за $O(\log n)$ времени (в параллельной модели).

Модель PRAM

Определение

PRAM (Parallel Random Access Machine) — идеализированная модель параллельных вычислений:

- есть **общая глобальная память**;
- есть (теоретически) **неограниченное число процессоров**, каждый процессор — RAM с собственным номером $p = 1, 2, \dots$;
- процессоры работают **синхронно**: за единицу времени выполняют одну инструкцию.

Разрешение конфликтов доступа к памяти

Для чтения/записи различают режимы **concurrent/exclusive**:

- **concurrent read** — несколько процессоров могут одновременно читать одну ячейку;
- **concurrent write** — несколько процессоров могут одновременно писать в одну ячейку; в модели CRCW нужно отдельно задавать правило разрешения конфликта (например, *arbitrary*: остаётся одно из записанных значений);
- **exclusive** — если несколько процессоров обращаются к одной ячейке "одновременно", это запрещено (поведение не определено/"мусор").

Стандартные варианты PRAM:

$$\text{CRCW PRAM}, \quad \text{CREW PRAM}, \quad \text{ERCW PRAM}, \quad \text{EREW PRAM}.$$

Класс NC

NC — класс задач, которые решаются на PRAM за время $(\log n)^{O(1)}$ при использовании *полиномиального* числа процессоров $n^{O(1)}$, где n – размер входа.

Как в PRAM посчитать `scan` на p процессорах

Пусть дан массив $A[0..n-1]$ и p процессоров. Разобьём массив на блоки длины $m = n/p$ (для простоты считаем $p \mid n$).

1. Каждый процессор i считает **локальный scan** на своём блоке $A[im..(i+1)m-1]$ последовательно за время $O(m) = O(n/p)$. Обозначим сумму блока как T_i (последний элемент локального scan).

2. Считаем **scan по массиву блок-сумм** $T_0, T_1, \ldots, T_{p-1}$ за время $O(\log p)$. Пусть
$$P_i = \sum_{j=0}^{i-1} T_j$$
— сумма всех предыдущих блоков (смещение для блока i).

3. Каждый процессор прибавляет смещение P_i ко всем элементам своего блока (снова $O(n/p)$ времени).

Итого время:
$$O\left(\frac{n}{p} + \log p\right).$$
В частности, при $p = \Theta\left(\frac{n}{\log n}\right)$ получаем $O(\log n)$ времени.

Пример : Сортировка подсчётом (Counting sort) в PRAM

Дан массив $a[0..n-1]$, причём $a[k] \in \{0, \ldots, U\}$, где $U = O(1)$. Есть p процессоров, нумеруем их $i = 0, 1, \ldots, p-1$. Каждый процессор обрабатывает блок длины $m = \frac{n}{p}$.

- Процессор i строит таблицу частот $c_i[0..U]$, где
$$c_i(x) = \#\{\, k \in [im, (i+1)m-1] \mid a[k] = x \,\}, \qquad m = \frac{n}{p}.$$
Это занимает $O(\frac{n}{p})$ времени.
- Далее соберём все частоты в один массив длины $(U+1)p$ в порядке
$$C[xp+i] = c_i(x), \qquad x = 0, \ldots, U, \quad i = 0, \ldots, p-1,$$
то есть
$$C = \big(c_0(0), c_1(0), \ldots, c_{p-1}(0),\ c_0(1), \ldots, c_{p-1}(1),\ \ldots,\ c_{p-1}(U)\big).$$
Посчитаем `scan` (префиксные суммы) по C и получим массив P.
- После этого процессор i знает, сколько элементов значения x находится у процессоров с номерами $\le i$ (это $P[xp+i]$), а значит знает и **диапазон позиций** в выходном массиве для своих элементов значения x:
$$\text{элементы } x \text{ из блока } i \text{ кладём в } \big[\, P[xp+i] - c_i(x),\ P[xp+i) \,\big)$$
И просто выписываем элементы в эти позиции.

Время. Локальный подсчёт частот занимает $O(n/p)$. `scan` по массиву длины $(U+1)p$ занимает $O(\log((U+1)p)) = O(\log p)$, так как $U = O(1)$. Итого время:

$$O\left(\frac{n}{p} + \log p\right).$$

4. TODO

5. Ранжирование списка во внешней памяти. Линейное программирование

5.1. Задача

Дан список, хранящийся в массиве $A[0..N-1]$ в произвольном порядке. У каждого узла x есть поля

$$x.\texttt{next}, \qquad x.w.$$

Также в реализации удобно хранить поле $x.\texttt{id}$, причём изначально

$$A[i].\texttt{id} = i.$$

Замечание

Ниже в рассуждениях мы иногда будем писать $\mathrm{pred}(x)$ для предшественника вершины x в текущем списке. Это *не поле структуры данных*, а только удобное математическое обозначение для анализа и для псевдокода.

Исходная задача состоит в том, чтобы для каждой вершины x определить её номер в списке, считая от головы:

$$1, 2, \ldots, N.$$

Эквивалентно, нужно упорядочить вершины массива A в порядке их следования вдоль списка.

Для решения этой задачи удобно рассмотреть более общую подзадачу: для каждой вершины x вычислить сумму весов на префиксе списка до неё, то есть

$$\mathrm{pref}(x) = \sum_{y \text{ от головы списка до } x} y.w.$$

После работы алгоритма в поле $x.w$ будет записано значение $\mathrm{pref}(x)$.

Если изначально у всех вершин $w = 1$, то $\mathrm{pref}(x)$ есть просто номер вершины x в списке, считая от головы:

$$1, 2, \ldots, N.$$

5.2. Идея сжатия списка

Пусть S — множество удаляемых узлов, такое что никакие два соседних по списку узла не удаляются одновременно:

$$\forall x \in S \quad \mathrm{pred}(x) \notin S, \qquad |S| = \Theta(N).$$

Тогда все узлы из S можно удалить *независимо друг от друга*: для каждого $x \in S$ выполняем

$$\mathrm{pred}(x).\texttt{next} = x.\texttt{next}, \qquad \mathrm{pred}(x).w \mathrel{+}= x.w.$$

После этого каждый оставшийся узел хранит суммарный вес некоторого подряд идущего блока исходных вершин.

На получившемся более коротком списке мы рекурсивно решаем *ту же самую задачу*: вычисляем префиксные суммы этих объединенных весов.

После возврата из рекурсии удалённые вершины нужно восстановить и дописать для них ответы

5.3. Как выбрать множество S

Каждой вершине независимо присваиваем случайный бит

$$x.r \in \{0, 1\}, \qquad \Pr[x.r = 0] = \Pr[x.r = 1] = \frac{1}{2}.$$

После этого выбираем

$$x \in S \iff x.r = 1 \text{ и } \mathrm{pred}(x).r = 0.$$

То есть в S попадают вершины, которые являются “правым концом” шаблона 01 вдоль списка.

Почему это корректно

Если $x \in S$, то по определению $\mathrm{pred}(x).r = 0, \qquad x.r = 1.$

Значит, $\mathrm{pred}(x)$ не может одновременно лежать в S, потому что для этого у него должен был бы быть бит 1.

Следовательно, $\forall x \in S\colon\ \mathrm{pred}(x) \notin S$.

Размер множества S

Для внутренней вершины вероятность попасть в S равна

$$\Pr[x \in S] = \Pr[\mathrm{pred}(x).r = 0 \wedge x.r = 1] = \frac{1}{4}.$$

Поэтому $\mathbb{E}|S| \approx \frac{N}{4}$,

а после одного шага остаётся примерно $N - |S| \approx \frac{3N}{4}$ вершин.

5.4. Рекурсивная схема алгоритма

Следующий псевдокод — это *не* реализация во внешней памяти, а только удобная запись логики алгоритма, где разрешено использовать обозначение $\mathrm{pred}(x)$.

Algorithm 6 PrefixSums-Conceptual(A)

Require: Текущий список A
Ensure: Для каждой вершины x значение $x.w$ равно сумме весов на префиксе до x

```
if |A| ≤ 1 then
    return A
Для каждой вершины x ∈ A независимо выбрать случайный бит x.r ∈ {0, 1}
S = { x ∈ A | pred(x) существует, pred(x).r = 0, x.r = 1 }
                                        ▷ Сжимаем список, удаляя вершины из S
for all x ∈ S do
    p = pred(x)
    p.w = p.w + x.w
    p.next = x.next
A′ = A \ S
PrefixSums-Conceptual(A′)
                                        ▷ Восстанавливаем удалённые вершины
for all x ∈ S в порядке, обратном удалению do
    p = pred(x)
    p.next = x
    p.w = p.w − x.w                     ▷ теперь p.w = префиксная сумма до p
    x.w = x.w + p.w                     ▷ теперь x.w = префиксная сумма до x
return A
```

5.5. Реализация через сортировки во внешней памяти

Главная цель — заменить случайные обращения к указателям на несколько сортировок и линейных проходов.

Замечание

Ниже для краткости не выписана отдельная обработка головы списка, хвоста списка и значения NULL в поле `next`. В реальной реализации эти пограничные случаи нужно обрабатывать отдельно (например, специальными фиктивными вершинами или отдельными ветками в коде).

Храним идентификаторы

У каждого узла есть поле

$$x.\mathtt{id}, \qquad A[i].\mathtt{id} = i.$$

Это позволяет после перестановок и сортировок восстановить “исходную” вершину.

Algorithm 7 EM_PrefixSums(A)

Require: Массив вершин A, отсортированный по `id`
Ensure: Для каждой вершины x значение $x.w$ равно сумме весов на префиксе до x

```
if |A| ≤ 1 then
    return A
(Â, S) = Compress(A)                ▷ Â — сжатый список, S — удалённые вершины
Â = EM_PrefixSums(Â)                ▷ рекурсивно решаем ту же задачу на объединенных весах
A = Restore(Â, S, |A|)
return A
```

Сортируем по `next`

Пусть массив A отсортирован по `id`. Тогда строим

$$\widetilde{A} = \mathrm{sort}(\{x \in A\} \text{ по } x.\texttt{next}).$$

После такой сортировки запись $\widetilde{A}[i]$ стоит напротив той вершины, в которую она указывает. Поэтому при одновременном просмотре массивов A и $\widetilde{A}$ можно сопоставить вершину и её предшественника.

Находим удаляемые вершины и сразу обновляем предшественников

При линейном проходе по A и $\widetilde{A}$:

$$\text{если } \widetilde{A}[i].r = 0 \text{ и } A[i].r = 1, \text{ то удаляем } A[i].$$

При этом выполняются обновления:

$$\widetilde{A}[i].w \mathrel{+}= A[i].w, \qquad \widetilde{A}[i].\texttt{next} = A[i].\texttt{next},$$

а сама удалённая вершина сохраняется в массив S.

Убираем “дубликаты” после перенаправления указателей

После удаления вершины её предшественник начинает указывать в того же преемника, в которого раньше указывала удалённая вершина. Поэтому после повторной сортировки по `next`

$$\widetilde{A} = \mathrm{sort}(\widetilde{A} \text{ по } \texttt{next})$$

появляются соседние элементы с одинаковым полем `next`.

Из каждой такой пары надо оставить только живую вершину, а удалённую выбросить. Это делается линейным проходом по отсортированному массиву: если у двух соседних элементов одинаковый `next`, то удаляется тот, у которого $r = 1$.

После этого выжившие вершины упаковываются в плотный массив длины $N - |S|$.

Algorithm 8 COMPRESS(A)

Require: Массив $A[0..N-1]$, отсортированный по `id`
Ensure: Сжатый массив $\widehat{A}$ и массив удалённых вершин S

```
Для каждой вершины x ∈ A независимо выбрать случайный бит x.r ∈ {0, 1}
Ã = sort(A по x.next)
S = [ ]
    ▷ Находим вершины из шаблона 01 и сразу переносим их вес в предшественника
for i = 0 to N − 1 do
    if Ã[i].r = 0 and A[i].r = 1 then
        Ã[i].w = Ã[i].w + A[i].w
        Ã[i].next = A[i].next
        S.PUSH_BACK(A[i])
Ã = sort(Ã по x.next)
                                    ▷ Упаковываем только живые вершины
B = [ ]
i = 0
while i < N do
    if i + 1 < N and Ã[i].next = Ã[i + 1].next then
        if Ã[i].r = 1 then
            swap(Ã[i], Ã[i + 1])    ▷ после swap живая вершина стоит первой
        B.PUSH_BACK(Ã[i])
        i = i + 2
    else
        B.PUSH_BACK(Ã[i])
        i = i + 1
            ▷ Переопределяем next: старые id заменяются на новые позиции в плотном
massиве
B̃ = sort(B по x.next)
i = 0; j = 0; k = 0
while j < N − |S| do
    while k < |S| and i = S[k].id do
        i = i + 1
        k = k + 1
    B̃[j].next = j
    i = i + 1
    j = j + 1
Â = sort(B̃ по x.id)        ▷ id сохраняются исходными, веса w уже агрегированы
return (Â, S)
```

Перенумерация и рекурсия

После упаковки нужно переопределить индексы `next` под новый массив, затем отсортировать вершины по `id` и рекурсивно решить задачу уже для массива размера $N - |S|$:

$$\widehat{A} = \mathrm{sort}(\{x \in A[0..N - |S| - 1]\} \text{ по } x.\texttt{id}).$$

Восстановление

Предполагается, что:

- S хранится в порядке возрастания `id`;

- $\widehat{A}$ — результат рекурсивного решения на сжатом массиве длины $N - |S|$.

Сначала сортируем удалённые и оставшиеся вершины по полю `next`:

$$\widetilde{S} = \mathrm{sort}(\{x \in S\} \text{ по } x.\texttt{next}), \qquad \widetilde{A} = \mathrm{sort}(\{x \in \widehat{A}[0..N - |S| - 1]\} \text{ по } x.\texttt{next}).$$

После этого выполняем линейный проход:

Algorithm 9 RESTORE($\widehat{A}, S, N$)

```
1: S̃ = sort(S по x.next)
2: Ã = sort(Â[0..N − |S| − 1] по x.next)
        ▷ i идёт по старым id, k — по S в порядке id    ▷ j идёт по живым вершинам в порядке next, m — по S в порядке next
3: i = 0; j = 0; k = 0; m = 0
4: while j < N − |S| do
5:     while k < |S| and i = S[k].id do
6:         i = i + 1
7:         k = k + 1                                    ▷ пропускаем старые id удалённых вершин
8:     Ã[j].next = i        ▷ восстановили старый id вершины, на которую вёл указатель
9:     if m < |S| and Ã[j].next = S̃[m].next then    ▷ значит, между предшественником и его текущим преемником надо вернуть удалённую вершину
10:        Ã[j].next = S̃[m].id
11:        Ã[j].w = Ã[j].w − S̃[m].w                    ▷ теперь Ã[j].w = префиксная сумма до предшественника
12:        S̃[m].w = S̃[m].w + Ã[j].w                    ▷ теперь S̃[m].w = префиксная сумма до восстановленной вершины
13:        m = m + 1
14:    i = i + 1
15:    j = j + 1
16: A = sort(x ∈ (S̃ ∪ Ã[0..N − |S| − 1]) по id)
17: return A
```

5.6. Оценка сложности

Обозначим через $\mathrm{sort}(N)$ стоимость сортировки N записей во внешней памяти.

Один уровень рекурсии состоит из:

- константного числа сортировок по полям `id` и `next`;
- константного числа линейных проходов по массивам.

Поэтому существует константа $c > 0$ такая, что работа одного уровня рекурсии не превосходит

$$c\ \mathrm{sort}(N).$$

Доказательство по индукции

Докажем по индукции, что

$$\mathbb{E}[T(n)] \le 4c\ \mathrm{sort}(n).$$

База индукции для малых n очевидна, если увеличить константу c при необходимости.

Пусть теперь утверждение верно для всех размеров меньше n. Тогда, раскладывая по возможным значениям $|S|$, получаем

$$\mathbb{E}[T(n)] = \sum_{k=0}^{n-1} \Pr[|S| = k] \cdot \mathbb{E}[T(n-k)] + c\,\mathrm{sort}(n).$$

Здесь для простоты суммируем по всем $k < n$, включая $k = 0$.

По предположению индукции:

$$\mathbb{E}[T(n)] \le \sum_{k=0}^{n-1} \Pr[|S| = k] \cdot 4c\,\mathrm{sort}(n-k) + c\,\mathrm{sort}(n).$$

Используем стандартную оценку

$$\mathrm{sort}(m) = \Theta\Big(\frac{m}{B}\log_{M/B}\frac{m}{B}\Big).$$

Тогда, поскольку $n - k \le n$,

$$\mathrm{sort}(n-k) \le \frac{n-k}{B}\log_{M/B}\frac{n}{B}$$

с точностью до постоянного множителя, уже поглощённого константой c. Следовательно,

$$\mathbb{E}[T(n)] \le 4c\log_{M/B}\frac{n}{B}\sum_{k=0}^{n-1}\Pr[|S| = k]\frac{n-k}{B} + c\,\mathrm{sort}(n).$$

Но

$$\sum_{k=0}^{n-1}\Pr[|S| = k](n-k) = \mathbb{E}[n - |S|].$$

Так как каждая внутренняя вершина удаляется с вероятностью $1/4$, в среднем после сжатия остаётся не более $\frac{3n}{4}$ вершин (с точностью до констант, связанных с крайними вершинами списка), то есть

$$\mathbb{E}[n - |S|] \le \frac{3n}{4}.$$

Подставляя это в предыдущую оценку, получаем

$$\mathbb{E}[T(n)] \le 4c\log_{M/B}\frac{n}{B}\cdot\frac{3n}{4B} + c\,\mathrm{sort}(n) \le 3c\,\mathrm{sort}(n) + c\,\mathrm{sort}(n) = 4c\,\mathrm{sort}(n).$$

Индукционный переход доказан, значит

$$\mathbb{E}[T(n)] = O(\mathrm{sort}(n)).$$

Итог

Таким образом, ожидаемая сложность алгоритма равна

$$\mathbb{E}[T(N)] = O(\mathrm{sort}(N)) = O\left(\frac{N}{B}\log_{M/B}\frac{N}{B}\right).$$

Предложение 5.1

Пусть дано корневое дерево на N вершинах. У каждой вершины v хранится её список детей в нужном порядке обхода:

```
struct { int id; int kids_num; int kids[kids_num]; }.
```

Требуется выписать все вершины в порядке обхода DFS.

Идея. Построим список, соответствующий эйлерову обходу дерева, и посчитаем на нём префиксные суммы весов.

Построение списка

Для каждого ребра (v, u), где u — ребёнок v, создаём две записи:

$$(v \to u) \qquad \text{и} \qquad (u \to v).$$

Также добавляем фиктивную стартовую запись s.

Поле `next` задаём так, чтобы эти записи шли в порядке эйлерова обхода:

- после дуги $(v \to u)$ идём либо в первую дугу из u к его первому ребёнку, либо, если детей нет, сразу в $(u \to v)$;
- после дуги $(u \to v)$ идём либо к следующему ребёнку вершины v, либо возвращаемся вверх.

Таким образом получаем список длины $2N - 1$.

Веса

Положим

$$s.w = 1,$$

$$(v \to u).w = 1 \quad \text{для дуги вниз,} \qquad (u \to v).w = 0 \quad \text{для дуги вверх.}$$

Теперь запускаем алгоритм вычисления префиксных сумм на этом списке.

Почему это работает

Единица добавляется ровно в момент первого входа в новую вершину. Поэтому:

- корень получает номер 1;
- для каждой дуги вниз $(v \to u)$ значение префиксной суммы на этой записи равно номеру вершины u в порядке DFS.

Значит, если собрать пары

$$(1, \text{root}) \qquad \text{и} \qquad ((v \to u).w,\ u) \text{ для всех дуг вниз,}$$

а затем отсортировать их по первому компоненту, то вершины будут выписаны в порядке обхода DFS.

Сложность

Число записей равно $O(N)$. Построение эйлерова списка и финальная сортировка требуют константного числа сортировок и линейных проходов, а префиксные суммы на списке считаются за $O(\mathrm{sort}(N))$.

Следовательно, вся задача решается за

$$O(\mathrm{sort}(N)).$$

Линейное программирование

5.7. Общая постановка задачи ЛП

Линейное программирование (ЛП) — это задача оптимизации линейной функции при линейных ограничениях.

В общем виде задача может быть записана так:

$$\max_{x \in \mathbb{R}^n} c^T x \qquad \text{при} \qquad Ax \le b,$$

где

$$A \in \mathbb{R}^{m \times n}, \qquad b \in \mathbb{R}^m, \qquad c \in \mathbb{R}^n.$$

Неравенство $Ax \le b$ понимается **покомпонентно**, то есть

$$(Ax)_i \le b_i, \qquad i = 1, \dots, m.$$

То же самое в координатной записи:

$$\max\ c_1 x_1 + \dots + c_n x_n$$

при условиях

$$\begin{cases} a_{11}x_1 + \dots + a_{1n}x_n \le b_1, \\ a_{21}x_1 + \dots + a_{2n}x_n \le b_2, \\ \qquad\qquad \vdots \\ a_{m1}x_1 + \dots + a_{mn}x_n \le b_m. \end{cases}$$

Две наиболее употребительные записи:

$$\max\ c^T x \quad \text{при } Ax \le b,$$

— *стандартная форма*, и

$$\min\ c^T x \quad \text{при } Ax = b,\ x \ge 0,$$

— *каноническая форма.*

Обе формы эквивалентны общей задаче после несложных преобразований.

5.8. Эквивалентные преобразования задач ЛП

Многие задачи линейного программирования можно привести к одной удобной форме.

1) Минимизацию можно заменить максимизацией

$$\min\ c^T x \quad \Longleftrightarrow \quad \max\ (-c)^T x.$$

2) Ограничение вида $\geq$ можно заменить ограничением вида $\leq$ Если

$$a_k^T x \geq b_k,$$

то, умножив обе части на -1, получаем

$$-a_k^T x \leq -b_k.$$

3) Равенство можно заменить двумя неравенствами Если

$$a_k^T x = b_k,$$

то это эквивалентно системе

$$\begin{cases} a_k^T x \leq b_k, \\ -a_k^T x \leq -b_k. \end{cases}$$

4) Свободную переменную можно заменить разностью двух неотрицательных Если переменная x_j может быть любого знака, то вводят

$$x_j^+ \geq 0, \qquad x_j^- \geq 0,$$

и полагают

$$x_j = x_j^+ - x_j^-.$$

Это стандартный способ перейти к задаче с неотрицательными переменными.

5) Неравенство $\leq$ можно превратить в равенство Если имеется ограничение

$$a_k^T x \leq b_k,$$

то вводится **добавочная переменная** (slack variable)

$$s_k \geq 0,$$

такая что

$$a_k^T x + s_k = b_k.$$

Смысл переменной s_k: это *остаток ресурса* или *запас* до границы ограничения.

5.9. Приведение к канонической форме

Одна из самых удобных форм:

$$\min\ c^T x \qquad \text{при} \qquad Ax = b, \qquad x \geq 0.$$

Чтобы привести к ней произвольную линейную задачу, обычно делают так:

1. при необходимости заменяют максимум на минимум или наоборот;
2. ограничения вида $\geq$ заменяют на $\leq$, умножая на -1;
3. неравенства заменяют равенствами с помощью добавочных переменных;
4. каждую свободную переменную заменяют разностью двух неотрицательных.

5.10. Геометрическая интерпретация

Пусть строки матрицы A равны

$$a_1^T, \dots, a_m^T.$$

Тогда каждое ограничение

$$a_k^T x \le b_k$$

задаёт **полупространство**.

Следовательно, допустимое множество

$$P = \{x \in \mathbb{R}^n \mid Ax \le b\}$$

есть пересечение полупространств, то есть **выпуклый многогранник**.

Целевая функция

$$c^T x$$

задаёт семейство гиперплоскостей

$$c^T x = d.$$

При изменении d эта гиперплоскость сдвигается параллельно самой себе. Задача ЛП состоит в том, чтобы найти такую гиперплоскость, которая:

1. ещё пересекает допустимое множество;
2. при этом даёт наибольшее (или наименьшее) значение целевой функции.

Пример : План производства

Предприятие выпускает два вида продукции:

$$x_1 = \text{число столов}, \qquad x_2 = \text{число стульев}.$$

Прибыль:

$$40 \text{ у.е. за стол}, \qquad 30 \text{ у.е. за стул}.$$

Есть два ресурса:

- древесина: не более 18 единиц;
- рабочее время: не более 10 часов.

На производство одной единицы продукции требуется:

	стол (x_1)	стул (x_2)
древесина	3	2
время	2	1

Тогда задача ЛП имеет вид:

$$\max\ 40x_1 + 30x_2$$

при ограничениях

$$\begin{cases} 3x_1 + 2x_2 \le 18, \\ 2x_1 + x_2 \le 10, \\ x_1 \ge 0,\ x_2 \ge 0. \end{cases}$$

Смысл ограничений:

- $3x_1 + 2x_2 \le 18$ — нельзя израсходовать больше древесины, чем есть;
- $2x_1 + x_2 \le 10$ — нельзя превысить имеющийся запас рабочего времени;
- $x_1, x_2 \ge 0$ — отрицательное число изделий производить нельзя.

5.11. Приближённое решение системы $Ax \approx b$

Иногда систему

$$Ax = b$$

нельзя решить точно: например, она переопределена или содержит ошибки измерений. Тогда ищут x, для которого невязка (ошибка)

$$Ax - b$$

как можно меньше.

Рассмотрим две постановки:

$$\min_x \|Ax - b\|_2 \qquad \text{и} \qquad \min_x \|Ax - b\|_1.$$

Норма ℓ_2: метод наименьших квадратов

Задача

$$\min_x \|Ax - b\|_2$$

имеет тот же набор решений, что и задача

$$\min_x \|Ax - b\|_2^2,$$

потому что квадрат — монотонная функция на $[0, \infty)$.

Именно для **квадрата нормы** выводятся **нормальные уравнения**:

$$A^T Ax = A^T b.$$

Почему получаются нормальные уравнения?

Рассмотрим функцию

$$f(x) = \|Ax - b\|_2^2 = (Ax - b)^T (Ax - b).$$

Необходимое условие минимума: все частные производные по переменным $x_1, \ldots, x_n$ равны нулю.

Вычисление этих производных приводит к равенству

$$2A^T(Ax - b) = 0.$$

Следовательно,

$$A^T(Ax - b) = 0,$$

то есть

$$A^T Ax = A^T b.$$

Это и есть нормальные уравнения.

Замечание. Если столбцы матрицы A линейно независимы, то матрица $A^T A$ невырождена, и решение единственно.

Оценка сложности для плотной матрицы $A \in \mathbb{R}^{m \times n}$:

- вычисление $A^T A$: $O(mn^2)$;
- вычисление $A^T b$: $O(mn)$;
- решение системы размера $n \times n$: $O(n^3)$;
- память: $O(n^2)$ для хранения $A^T A$.

Норма ℓ_1: сведение к линейному программированию

Задача

$$\min_x \|Ax - b\|_1$$

может быть сведена к задаче линейного программирования.

Напомним, что

$$\|Ax - b\|_1 = \sum_{i=1}^{m} |(Ax - b)_i|.$$

Чтобы убрать модуль, вводят новые переменные

$$t_1, \ldots, t_m \geq 0,$$

где каждая t_i должна быть не меньше абсолютной величины соответствующей невязки:

$$|(Ax - b)_i| \leq t_i.$$

Это эквивалентно двум линейным ограничениям:

$$-(t_i) \leq (Ax - b)_i \leq t_i.$$

Значит, задача переписывается так:

$$\min_{x,t} \sum_{i=1}^{m} t_i$$

при условиях

$$Ax - b \leq t, \qquad -(Ax - b) \leq t, \qquad t \geq 0.$$

Эквивалентно:

$$\min_{x,t} \mathbf{1}^T t$$

при

$$\begin{cases} Ax - t \leq b, \\ -Ax - t \leq -b, \\ t \geq 0. \end{cases}$$

Почему это работает На оптимуме каждая переменная t_i станет минимально возможной, то есть

$$t_i = |(Ax - b)_i|.$$

Поэтому целевая функция

$$\sum_{i=1}^{m} t_i$$

ровно совпадает с $\|Ax - b\|_1$.

Размер новой задачи

- новых переменных: m;
- новых линейных ограничений: $2m$;
- стоимость построения для плотного случая: $O(mn)$.

Пример : для ℓ_1

Пусть
$$A = \begin{pmatrix} 1 \\ 2 \end{pmatrix}, \qquad b = \begin{pmatrix} 1 \\ 4 \end{pmatrix}.$$
Тогда
$$Ax - b = \begin{pmatrix} x - 1 \\ 2x - 4 \end{pmatrix},$$
и задача
$$\min_x \|Ax - b\|_1$$
имеет вид
$$\min_x \big(|x - 1| + |2x - 4|\big).$$
Введём $t_1, t_2 \geq 0$. Тогда получаем ЛП:
$$\min_{x, t_1, t_2} t_1 + t_2$$
при условиях
$$\begin{cases} x - 1 \leq t_1, \\ -(x - 1) \leq t_1, \\ 2x - 4 \leq t_2, \\ -(2x - 4) \leq t_2, \\ t_1 \geq 0, \ t_2 \geq 0. \end{cases}$$
Это уже обычная задача линейного программирования без модулей.

Предложение 5.2

Как решить $\min c^T x$ при $Ax = b$, если переменные свободны по знаку

Идея решения

Если уже есть алгоритм, который умеет решать задачи вида
$$\min \ \tilde{c}^T z \qquad \text{при} \qquad \tilde{A} z = \tilde{b}, \qquad z \geq 0,$$
то нужно только избавиться от свободных переменных.

Для каждого $j = 1, \ldots, n$ вводим
$$x_j = x_j^+ - x_j^-, \qquad x_j^+ \geq 0, \quad x_j^- \geq 0.$$
Тогда
$$x = x^+ - x^-,$$
и исходная задача эквивалентна
$$\min \ c^T (x^+ - x^-)$$

при

$$A(x^+ - x^-) = b, \qquad x^+ \geq 0, \quad x^- \geq 0.$$

В матричном виде:

$$\min \begin{pmatrix} c \\ -c \end{pmatrix}^T \begin{pmatrix} x^+ \\ x^- \end{pmatrix}$$

при

$$\begin{pmatrix} A & -A \end{pmatrix} \begin{pmatrix} x^+ \\ x^- \end{pmatrix} = b, \qquad \begin{pmatrix} x^+ \\ x^- \end{pmatrix} \geq 0.$$

Это уже задача в канонической форме.

Размер новой задачи

- было n переменных, стало $2n$;
- число равенств осталось m.

Построение новой задачи требует $O(mn)$ операций для плотной матрицы.

6. TODO

7. Линейное программирование и метод внутренней точки

Рассматривается стандартная пара задач линейного программирования

$$\mathcal{P}: \quad \max\ c^\top x \quad \text{при условиях} \quad Ax \leq b,\ x \geq 0,$$

$$\mathcal{D}: \quad \min\ b^\top y \quad \text{при условиях} \quad A^\top y \geq c,\ y \geq 0,$$

где $A \in \mathbb{Q}^{m \times n}$, $b \in \mathbb{Q}^m$, $c \in \mathbb{Q}^n$.

Предложение 7.1: Слабая двойственность

Для любых допустимых решений x прямой задачи и y двойственной задачи выполнено

$$c^\top x \leq b^\top y.$$

Доказательство. Из $A^\top y \geq c$ и $x \geq 0$ получаем

$$c^\top x \leq (A^\top y)^\top x = y^\top A x.$$

Далее, из $Ax \leq b$ и $y \geq 0$ следует

$$y^\top A x \leq y^\top b = b^\top y.$$

Следовательно,

$$c^\top x \leq b^\top y.$$

■

Замечание : Сведение ЛП к задаче выполнимости

Предположим, что имеется алгоритм, который за полиномиальное время решает задачу выполнимости системы линейных неравенств: либо находит решение, либо сообщает, что решений нет.
Тогда задачу линейного программирования можно свести к выполнимости одной системы линейных неравенств. Действительно, x и y являются оптимальными решениями задач $\mathcal{P}$ и $\mathcal{D}$ тогда и только тогда, когда выполнена система

$$Ax \le b, \qquad x \ge 0, \qquad A^\top y \ge c, \qquad y \ge 0, \qquad b^\top y - c^\top x \le 0.$$

По слабой двойственности для любых допустимых x и y всегда выполнено

$$b^\top y - c^\top x \ge 0.$$

Следовательно, добавленное неравенство $b^\top y - c^\top x \le 0$ эквивалентно равенству

$$b^\top y = c^\top x,$$

то есть условию оптимальности. Таким образом, нахождение оптимального решения сводится к нахождению решения системы линейных неравенств.

Предложение 7.2: Задачи $\mathcal{P}$ и $\mathcal{D}$ образуют двойственную пару

Задача $\mathcal{D}$ является двойственной к задаче $\mathcal{P}$, а задача $\mathcal{P}$ является двойственной к задаче $\mathcal{D}$.

Доказательство. Пусть x — допустимое решение задачи $\mathcal{P}$, а y удовлетворяет условиям

$$A^\top y \ge c, \qquad y \ge 0.$$

Тогда из $x \ge 0$ получаем

$$c^\top x \le (A^\top y)^\top x = y^\top A x.$$

Так как $Ax \le b$ и $y \ge 0$, имеем

$$y^\top A x \le y^\top b = b^\top y.$$

Следовательно,

$$c^\top x \le b^\top y.$$

Итак, любой вектор y, удовлетворяющий условиям

$$A^\top y \ge c, \qquad y \ge 0,$$

задаёт верхнюю оценку $b^\top y$ на значение целевой функции прямой задачи. Поэтому среди всех таких оценок естественно выбирать наименьшую, то есть решать задачу

$$\min\ b^\top y \quad \text{при условиях} \quad A^\top y \ge c,\ y \ge 0.$$

Это и есть задача $\mathcal{D}$.

Аналогично, начиная с задачи $\mathcal{D}$, получаем, что двойственная к ней задача имеет вид

$$\max\ c^\top x \quad \text{при условиях} \quad Ax \le b,\ x \ge 0,$$

то есть совпадает с $\mathcal{P}$. ■

7.1. Длина входа и полиномиальность

Определение : Длина входа

Пусть все элементы матрицы A, вектора b и вектора c являются рациональными числами, каждое из которых кодируется не более чем w битами. Тогда длиной входа задачи линейного программирования называется величина

$$L = (mn + m + n)\, w = \Theta(mnw).$$

Определение : Полиномиальный алгоритм

Алгоритм решения задачи линейного программирования называется *полиномиальным*, если его время работы оценивается величиной

$$O\big(L^{O(1)}\big).$$

7.2. Метод внутренней точки

Определение : Внутренняя точка

Точка x_0 называется *внутренней допустимой точкой* для системы $Ax \le b$, если

$$Ax_0 < b$$

покомпонентно.

Замечание : Как найти внутреннюю точку

Для поиска внутренней точки удобно рассмотреть вспомогательную задачу

$$\max\ t \quad \text{при условии} \quad Ax + t\mathbf{1} \le b,$$

где $\mathbf{1} = (1, \dots, 1)^\top \in \mathbb{R}^m$.
Если оптимальное значение t^* положительно, то соответствующая точка x^* удовлетворяет

$$Ax^* < b,$$

то есть является внутренней точкой исходной системы.
У вспомогательной задачи всегда есть очевидная внутренняя точка, например

$$x = 0, \qquad t = -\max_{1 \le i \le m} |b_i| - 1,$$

поскольку тогда $t < b_i$ для всех i, а значит

$$Ax + t\mathbf{1} = t\mathbf{1} < b.$$

Барьерная задача

Пусть строки матрицы A имеют вид $a_1^\top, \dots, a_m^\top$, а

$$s_i(x) := b_i - a_i^\top x$$

обозначают зазоры до ограничений.

Определение : Логарифмический барьер

Для x, лежащих во внутренности многогранника $Ax < b$, положим

$$g(x) := \sum_{i=1}^{m} \ln s_i(x) = \sum_{i=1}^{m} \ln\bigl(b_i - a_i^\top x\bigr).$$

Для параметра $w > 0$ рассмотрим барьерную задачу

$$\max\ \bigl(w\, c^\top x + g(x)\bigr) \quad \text{при условии} \quad Ax \le b.$$

Обозначим через x_w^* одно из её решений.

Предложение 7.3: Центральный путь

Семейство точек $\{x_w^*\}_{w>0}$ образует *центральный путь*. Кроме того,

$$\arg\max_{Ax<b}\bigl(w\, c^\top x + g(x)\bigr) = \arg\max_{Ax<b}\left(c^\top x + \frac{1}{w} g(x)\right).$$

Поэтому при $w \to +\infty$ влияние барьерного слагаемого ослабевает, и точки x_w^* приближаются к оптимальному решению исходной задачи линейного программирования.

Замечание : Геометрический смысл

Барьерная функция $g(x)$ стремится к $-\infty$ при приближении к границе многогранника $Ax \le b$. Поэтому максимизатор x_w^* остаётся во внутренности допустимой области. При увеличении w эти точки движутся внутри многогранника в сторону оптимального решения исходной задачи.

Идея алгоритма

Пусть

$$f_k(x) := w_k\, c^\top x + g(x),$$

где $0 < w_0 < w_1 < w_2 < \cdots$. Идея метода состоит в том, чтобы выбирать параметры w_k возрастающими и поддерживать точку x_k, которая достаточно близка к центральной точке

$$x_k^* := x_{w_k}^*.$$

После этого из x_k выполняется один шаг Ньютона, который перемещает точку ближе к x_{k+1}^*.

Шаг Ньютона

Замечание : Обозначения ∇f и $\nabla^2 f$

Пусть $f(x_1, \ldots, x_n)$ — функция многих переменных.
Символ

$$\nabla f(x)$$

обозначает *градиент* функции f в точке x, то есть столбец её первых частных производных:

$$\nabla f(x) = \begin{pmatrix} \frac{\partial f}{\partial x_1}(x) \\ \vdots \\ \frac{\partial f}{\partial x_n}(x) \end{pmatrix}.$$

Символ

$$\nabla^2 f(x)$$

обозначает *гессиан* функции f в точке x, то есть матрицу её вторых частных производных:

$$\nabla^2 f(x) = \left(\frac{\partial^2 f}{\partial x_i \partial x_j}(x) \right)_{i,j=1}^{n}.$$

То есть знак ∇ — это просто обозначение для набора первых производных, а знак ∇^2 — для набора вторых производных.

Предложение 7.4: Градиент и гессиан барьерной функции

Для функции

$$g(x) = \sum_{i=1}^{m} \ln\bigl(b_i - a_i^\top x\bigr)$$

справедливы формулы

$$\nabla g(x) = -\sum_{i=1}^{m} \frac{a_i}{s_i(x)}, \qquad \nabla^2 g(x) = -\sum_{i=1}^{m} \frac{a_i a_i^\top}{s_i(x)^2}.$$

Следовательно,

$$\nabla f_k(x) = w_k c - \sum_{i=1}^{m} \frac{a_i}{s_i(x)}, \qquad \nabla^2 f_k(x) = -\sum_{i=1}^{m} \frac{a_i a_i^\top}{s_i(x)^2}.$$

Доказательство. Для каждого i имеем

$$\nabla \ln\bigl(b_i - a_i^\top x\bigr) = -\frac{a_i}{b_i - a_i^\top x} = -\frac{a_i}{s_i(x)}.$$

Повторное дифференцирование даёт

$$\nabla^2 \ln\bigl(b_i - a_i^\top x\bigr) = -\frac{a_i a_i^\top}{\bigl(b_i - a_i^\top x\bigr)^2} = -\frac{a_i a_i^\top}{s_i(x)^2}.$$

Остаётся просуммировать по $i = 1, \ldots, m$. ■

Замечание : Квадратическая аппроксимация

Во второй окрестности точки x_k функция f_k аппроксимируется формулой Тейлора второго порядка:

$$\widetilde{f}_k(x) = f_k(x_k) + \nabla f_k(x_k)^\top (x - x_k) + \frac{1}{2}(x - x_k)^\top \nabla^2 f_k(x_k)\,(x - x_k).$$

Точка максимума квадратической модели определяется из условия

$$\nabla \widetilde{f}_k(x) = 0,$$

то есть из линейной системы

$$0 = \nabla f_k(x_k) + \nabla^2 f_k(x_k)\,(x - x_k).$$

Отсюда получается шаг Ньютона

$$x_{k+1} = x_k - \left(\nabla^2 f_k(x_k)\right)^{-1} \nabla f_k(x_k).$$

Эллипсоид Дикина

Определение : Эллипсоид Дикина

Для точки z, удовлетворяющей $Az < b$, и радиуса $R > 0$ определим

$$\mathcal{E}(z, R) := \left\{ x \in \mathbb{R}^n : \sum_{i=1}^{m} \frac{\left(s_i(x) - s_i(z)\right)^2}{s_i(z)^2} \le R^2 \right\}.$$

Так как

$$s_i(x) - s_i(z) = -a_i^\top (x - z),$$

это равносильно записи

$$\mathcal{E}(z, R) = \left\{ x \in \mathbb{R}^n : (x - z)^\top \left(\sum_{i=1}^{m} \frac{a_i a_i^\top}{s_i(z)^2} \right) (x - z) \le R^2 \right\}.$$

Замечание : Назначение эллипсоида Дикина

Эллипсоид Дикина задаёт естественную локальную геометрию барьерной функции и служит удобной метрикой для измерения близости точки x_k к центральной точке x_k^*.

Схема алгоритма

Утверждение 7.1: Схема метода внутренней точки

Пусть

$$w_0 = 2^{-\Theta(L)}, \qquad w_{k+1} = \left(1 + \frac{1}{100\sqrt{m}}\right) w_k.$$

1. Инициализация: выбрать внутреннюю точку x_0.
2. Для $k = 0, 1, 2, \ldots$:

   положить

   $$f_k(x) = w_k c^\top x + g(x);$$

   выполнить шаг Ньютона

   $$x_{k+1} = x_k - \left(\nabla^2 f_k(x_k)\right)^{-1} \nabla f_k(x_k);$$

обновить параметр

$$w_{k+1} = \left(1 + \frac{1}{100\sqrt{m}}\right) w_k.$$

Утверждение 7.2: Типовые инварианты и оценки

В стандартном анализе метода внутренней точки доказываются следующие факты.

1. Если текущая точка удовлетворяет

$$x_k \in \mathcal{E}\left(x_k^*, \frac{1}{12}\right),$$

то после умеренного увеличения параметра w_k новая центральная точка x_{k+1}^* остаётся близкой к x_k^*.

2. Один шаг Ньютона, выполненный из точки, достаточно близкой к x_k^*, переносит итерацию в точку, которая достаточно близка к x_{k+1}^*.
3. В частности, можно поддерживать инвариант вида

$$x_k \in \mathcal{E}\left(x_k^*, \frac{1}{12}\right) \quad \text{для всех } k.$$

4. Для оптимального значения x^* исходной задачи верна оценка разрыва

$$\left|c^\top x_k - c^\top x^*\right| = O\left(\frac{m}{w_k}\right).$$

5. Поэтому достаточно увеличить w_k до величины порядка

$$w_k = \Theta\big(m\, 2^{\Theta(L)}\big),$$

чтобы обеспечить точность

$$\left|c^\top x_k - c^\top x^*\right| < 2^{-\Theta(L)}.$$

Предложение 7.5: Число итераций

Пусть

$$w_k = \left(1 + \frac{1}{100\sqrt{m}}\right)^k w_0, \qquad w_0 = 2^{-\Theta(L)}.$$

Тогда для достижения точности $2^{-\Theta(L)}$ достаточно

$$k = O(\sqrt{m}\, L)$$

итераций.

Доказательство. Необходимо довести w_k от величины $2^{-\Theta(L)}$ до величины $\Theta(m\, 2^{\Theta(L)})$. Это эквивалентно условию

$$\frac{w_k}{w_0} = m\, 2^{\Theta(L)}.$$

Беря логарифм, получаем

$$k \ln\left(1 + \frac{1}{100\sqrt{m}}\right) = \Theta(L + \ln m).$$

Так как $\ln(1+\varepsilon) = \Theta(\varepsilon)$ при малых ε, имеем

$$\ln\left(1 + \frac{1}{100\sqrt{m}}\right) = \Theta\left(\frac{1}{\sqrt{m}}\right).$$

Следовательно,

$$k = O\big(\sqrt{m}\,(L + \ln m)\big) = O(\sqrt{m}\,L).$$

■

Пример : Единичный ℓ_1-шар

Множество

$$\{x \in \mathbb{R}^n : |x_1| + \cdots + |x_n| \le 1\}$$

является выпуклым многогранником.

1. При $n = 2$ это ромб.
2. При $n = 3$ это октаэдр.

Именно такие фигуры были схематически изображены на доске как пример многогранников.

8. Потоки в сетях. Алгоритм Форда–Фалкерсона. Теорема о максимальном потоке и минимальном разрезе

Определение : Транспортная сеть

Пусть дан ориентированный граф $G = (V, E, c)$, где функция

$$c\colon E \to \mathbb{R}_{\ge 0}$$

задаёт пропускные способности рёбер.
Граф G называется *транспортной сетью*, если в нём выделены две вершины:

$$s \in V \quad \text{(исток)}, \qquad t \in V \quad \text{(сток)},$$

причём

$$\deg^-(s) = 0, \qquad \deg^+(t) = 0.$$

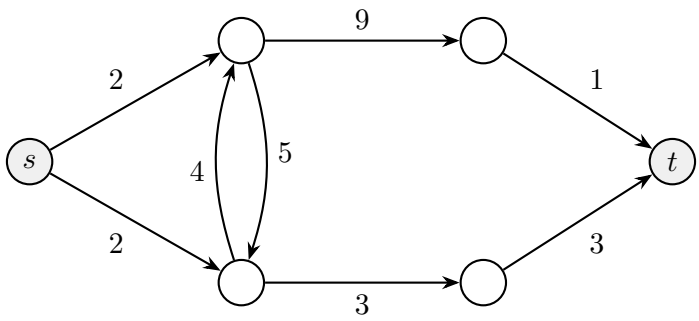


Рис. 15: Пример транспортной сети. Числа у рёбер — пропускные способности $c(u, v)$.

Определение : Поток

Потоком в транспортной сети называется функция $f\colon E \to \mathbb{R}_{\geq 0}$, такая что:

1) **Ограничение пропускной способности:**

$$0 \leq f(u, v) \leq c(u, v) \qquad \forall (u, v) \in E.$$

2) **Закон сохранения потока:** для любой вершины $u \in V \setminus \{s, t\}$ выполнено

$$\sum_{(v,u)\in E} f(v, u) = \sum_{(u,w)\in E} f(u, w).$$

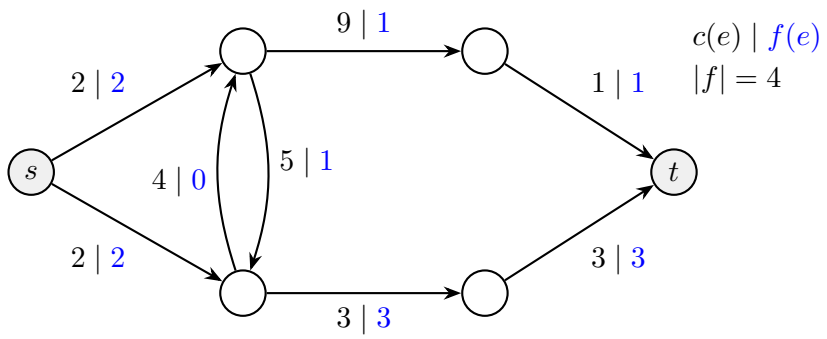


Рис. 16: Поток величины 4 в транспортной сети из рис. 15. Рядом с каждым ребром указана пара *пропускная способность* / *поток*.

Определение : Величиной потока

Величиной потока f называется число

$$|f| = \sum_{(s,v)\in E} f(s, v).$$

Эквивалентно,

$$|f| = \sum_{(u,t)\in E} f(u, t).$$

Предложение 8.1

Для любого потока f выполнено

$$\sum_{(s,v)\in E} f(s,v) = \sum_{(u,t)\in E} f(u,t).$$

Доказательство. Просуммируем закон сохранения потока по всем вершинам из множества $V \setminus \{s,t\}$. Все слагаемые, соответствующие внутренним рёбрам, сократятся попарно, и останется только суммарный поток, выходящий из истока, и суммарный поток, входящий в сток. ■

8.1. Задача о максимальном потоке

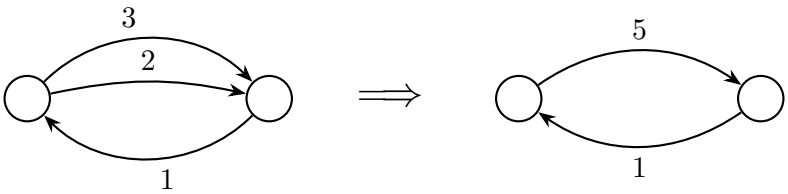
Требуется найти поток максимальной величины.

Замечание

Многие комбинаторные задачи допускают сведение к задаче о максимальном потоке.

Замечание

Будем считать, что в графе нет кратных рёбер. Это не ограничивает общность: несколько параллельных рёбер можно заменить одним ребром с суммарной пропускной способностью.



Предложение 8.2

Задача о максимальном потоке допускает формулировку в виде задачи линейного программирования:

$$\max \sum_{(s,v)\in E} f(s,v)$$

при ограничениях

$$0 \le f(u,v) \le c(u,v) \qquad \forall (u,v) \in E,$$

$$\sum_{(v,u)\in E} f(v,u) - \sum_{(u,w)\in E} f(u,w) = 0 \qquad \forall u \in V \setminus \{s,t\}.$$

Замечание

Множество допустимых потоков замкнуто и ограничено в $\mathbb{R}^E$, следовательно, компактно. Поэтому непрерывная функция

$$(f(e))_{e\in E} \mapsto \sum_{(s,v)\in E} f(s,v)$$

достигает на нем максимума по теореме Вейерштрасса.

8.2. Идея построения максимального потока

1) Начинаем с нулевого потока $f \equiv 0$.

2) На каждом шаге пытаемся увеличить поток, пропуская дополнительную порцию потока из s в t по некоторому допустимому пути.

Однако возникает существенная трудность: иногда для увеличения общего потока необходимо не только добавлять поток по прямым рёбрам, но и частично *отменять* ранее проведённый поток по некоторым рёбрам. Для формализации этого вводится остаточная сеть.

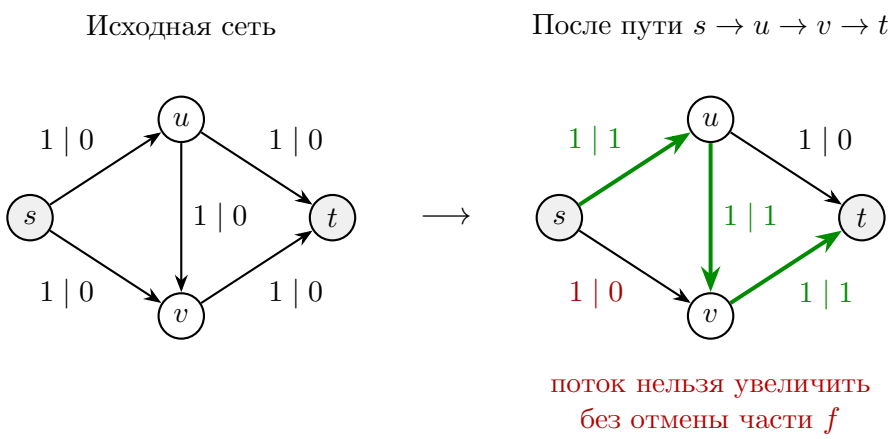


Рис. 17: Жадный выбор пути $s \to u \to v \to t$ насыщает ребро $v \to t$ и блокирует путь $s \to v \to t$. Для получения оптимального потока величины 2 необходимо «отменить» единицу потока через ребро $u \to v$ с помощью обратного ребра остаточной сети.

8.3. Дополняющие пути и остаточная сеть

Определение

Пусть f — поток в сети $G = (V, E, c)$. *Остаточной сетью* называется ориентированный граф

$$G_f = (V, E_f, c_f),$$

который строится следующим образом. Для каждого ребра $(u, v) \in E$:

1) если $c(u, v) - f(u, v) > 0$, то в E_f добавляется *прямое* ребро (u, v) остаточной пропускной способности

$$c_f(u, v) = c(u, v) - f(u, v);$$

2) если $f(u, v) > 0$, то в E_f добавляется *обратное* ребро (v, u) остаточной пропускной способности

$$c_f(v, u) = f(u, v).$$

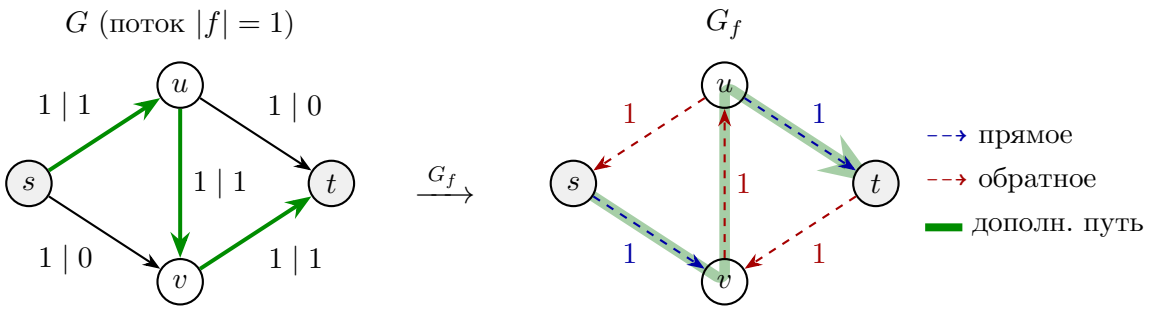


Рис. 18: Построение остаточной сети G_f. Синие пунктирные рёбра — прямые (можно добавить поток), красные пунктирные — обратные (можно отменить поток). Зелёным выделен дополняющий путь $s \to v \to u \to t$ в G_f, позволяющий увеличить поток до 2.

Замечание

В остаточной сети могут появляться кратные рёбра, если прямое и обратное направления возникают независимо. Это не влияет на корректность алгоритма.

Определение

Путь

$$P = (v_0 = s, v_1, \ldots, v_k = t)$$

в графе G_f называется *дополняющим путём* для потока f.

Определение

Пусть $P = (v_0 = s, v_1, \ldots, v_k = t)$ — дополняющий путь в G_f. Его *пропускной способностью* называется число

$$\delta(P) = \min_{0 \le i < k} c_f(v_i, v_{i+1}).$$

Предложение 8.3

Пусть $P = (v_0 = s, v_1, \ldots, v_k = t)$ — дополняющий путь в G_f, и $\delta = \delta(P)$. Тогда можно построить новый поток f' величины

$$|f'| = |f| + \delta.$$

Доказательство. Для каждого ребра пути действуем следующим образом:

1) если ребро (v_i, v_{i+1}) является прямым ребром исходной сети, то полагаем

$$f'(v_i, v_{i+1}) = f(v_i, v_{i+1}) + \delta;$$

2) если ребро (v_i, v_{i+1}) является обратным, то есть в исходной сети имеется ребро (v_{i+1}, v_i), то полагаем

$$f'(v_{i+1}, v_i) = f(v_{i+1}, v_i) - \delta.$$

На всех остальных рёбрах поток не меняется.

Поскольку $\delta \le c_f(v_i, v_{i+1})$ для каждого ребра пути, ограничения пропускной способности сохраняются. Закон сохранения потока также сохраняется: в каждой промежуточной вер-

шине изменение входящего потока компенсируется изменением исходящего потока на ту же величину δ.

Остаётся проверить, что величина потока увеличилась на δ. По определению величины потока

$$|f| = \sum_{(s,v)\in E} f(s,v).$$

Первое ребро дополняющего пути, выходящее из s, является прямым ребром исходной сети, поэтому на этом ребре поток увеличивается на δ. Другие изменения, связанные с вершиной s, отсутствуют. Следовательно,

$$|f'| = \sum_{(s,v)\in E} f'(s,v) = \sum_{(s,v)\in E} f(s,v) + \delta = |f| + \delta.$$

■

8.4. Алгоритм Форда–Фалкерсона

Утверждение 8.1

Алгоритм Форда–Фалкерсона строит поток итеративно следующим образом:

1) положить $f \equiv 0$;
2) построить остаточную сеть G_f;
3) найти в G_f путь P из s в t;
4) если такого пути нет, остановиться: текущий поток f является максимальным;
5) если путь P найден, увеличить поток вдоль него на величину $\delta(P)$, получив новый поток f;
6) перестроить остаточную сеть G_f для нового потока и перейти к шагу 3.

Замечание

На шаге поиска пути можно использовать как DFS, так и BFS. От выбора способа поиска существенно зависит итоговая оценка времени работы.

8.5. s–t-разрез

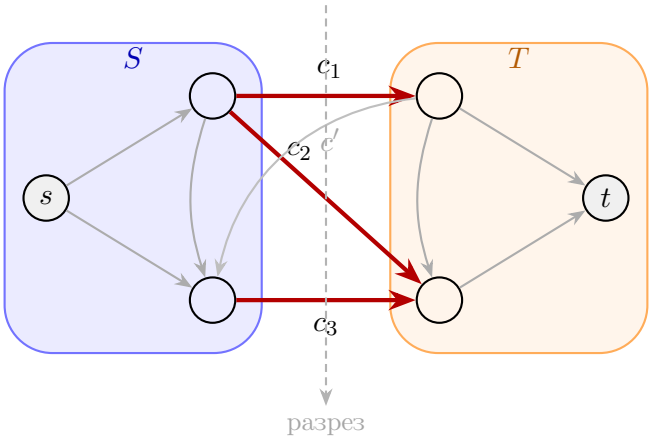
Определение

s–t-разрезом называется разбиение множества вершин $V = S \sqcup T, \quad s \in S, t \in T$.

Определение

Величиной разреза (S,T) называется сумма пропускных способностей всех рёбер, идущих из S в T:

$$c(S,T) = \sum_{\substack{(u,v)\in E \\ u\in S,\ v\in T}} c(u,v).$$



$$c(S,T) = c_1 + c_2 + c_3$$
(ребро $c' \colon T \to S$ не учитывается)

Рис. 19: s–t-разрез (S,T) и его величина $c(S,T)$. Красные рёбра идут из S в T и входят в сумму, а серое ребро из T в S не учитывается.

Предложение 8.4

Для любого потока f и любого s–t-разреза (S,T) выполнено неравенство

$$|f| \le c(S,T).$$

Доказательство. Рассмотрим сумму

$$\sum_{u \in S} \left(\sum_{(v,u) \in E} f(v,u) - \sum_{(u,w) \in E} f(u,w) \right).$$

Для всех вершин из $S \setminus \{s\}$ эта величина равна нулю по закону сохранения потока, а для вершины s она равна $-|f|$. После сокращения внутренних рёбер получаем

$$|f| = \sum_{\substack{(u,v) \in E \\ u \in S,\ v \in T}} f(u,v) - \sum_{\substack{(u,v) \in E \\ u \in T,\ v \in S}} f(u,v).$$

Следовательно,

$$|f| \le \sum_{\substack{(u,v) \in E \\ u \in S,\ v \in T}} f(u,v) \le \sum_{\substack{(u,v) \in E \\ u \in S,\ v \in T}} c(u,v) = c(S,T).$$

■

8.6. Теорема о максимальном потоке и минимальном разрезе

Теорема 8.1

Для потока f в транспортной сети эквивалентны следующие условия:

1) f является максимальным потоком;
2) в остаточной сети G_f нет дополняющего пути из s в t;

3) существует s–t-разрез (S, T) такой, что

$$|f| = c(S, T).$$

Доказательство. Докажем циклически

$$1) \Rightarrow 2) \Rightarrow 3) \Rightarrow 1).$$

$1) \Rightarrow 2)$**.** Если бы в G_f существовал дополняющий путь, то по предыдущему предложению поток можно было бы увеличить. Это противоречит максимальности f.

$2) \Rightarrow 3)$**.** Пусть S — множество вершин, достижимых из s в остаточной сети G_f, а

$$T = V \setminus S.$$

Тогда $s \in S$, а $t \notin S$, поскольку по предположению пути из s в t в G_f нет.

Покажем, что

$$|f| = c(S, T).$$

Из доказательства оценки для разреза имеем

$$|f| = \sum_{\substack{(u,v)\in E \\ u\in S,\ v\in T}} f(u, v) - \sum_{\substack{(u,v)\in E \\ u\in T,\ v\in S}} f(u, v).$$

Теперь используем определение множества S.

Если $(u, v) \in E$, $u \in S$, $v \in T$, то в остаточной сети не может быть прямого ребра (u, v), иначе вершина v была бы достижима из s. Значит,

$$c(u, v) - f(u, v) = 0, \qquad \text{то есть} \qquad f(u, v) = c(u, v).$$

Если $(u, v) \in E$, $u \in T$, $v \in S$, то в остаточной сети не может быть обратного ребра (v, u), иначе вершина u была бы достижима из s. Значит,

$$f(u, v) = 0.$$

Следовательно,

$$|f| = \sum_{\substack{(u,v)\in E \\ u\in S,\ v\in T}} c(u, v) = c(S, T).$$

$3) \Rightarrow 1)$**.** Для любого потока f' и любого разреза (S, T) верно

$$|f'| \le c(S, T).$$

Если для некоторого потока f выполнено равенство

$$|f| = c(S, T),$$

то никакой поток не может иметь величину больше $|f|$. Следовательно, f максимален. ■

Предложение 8.5

Максимальная величина потока равна минимальной величине s–t-разреза.

Доказательство. Теорема утверждает существование потока и разреза, для которых достигается равенство

$$|f| = c(S, T).$$

С другой стороны, для любого потока и любого разреза всегда выполнено неравенство

$$|f| \leq c(S, T).$$

Отсюда и следует утверждение. ■

8.7. Оценки времени работы

Предложение 8.6

Одна итерация алгоритма Форда–Фалкерсона работает за время

$$O(|E|),$$

если поиск дополняющего пути выполняется обходом графа в глубину или в ширину.

Доказательство. Построение остаточной сети и поиск пути в ней занимают линейное по числу рёбер время. ■

Предложение 8.7

Если все пропускные способности целочисленны, то алгоритм Форда–Фалкерсона завершится не более чем за

$$|f_{\max}|$$

итераций, где $f_{\max}$ — максимальный поток.

Доказательство. При целочисленных пропускных способностях каждая остаточная пропускная способность также целочисленна. Следовательно, на каждой итерации

$$\delta(P) \geq 1,$$

и величина потока увеличивается хотя бы на единицу. ■

Следствие

В случае целочисленных пропускных способностей время работы алгоритма Форда–Фалкерсона оценивается как

$$O(|E| \cdot |f_{\max}|).$$

Замечание

Для произвольных вещественных пропускных способностей алгоритм Форда–Фалкерсона при неудачном выборе дополняющих путей может работать очень долго; в общем случае возможны патологические сценарии.

Утверждение 8.2

Если на каждом шаге искать кратчайший по числу рёбер дополняющий путь с помощью BFS, то получается алгоритм Эдмондса–Карпа со временем работы

$$O(|V| \cdot |E|^2).$$

Замечание

Существуют и более быстрые алгоритмы решения задачи о максимальном потоке, в частности алгоритм Диница и алгоритм Орлина.

9. TODO

10. TODO

11. TODO

12. TODO

13. TODO

14. TODO

15. TODO